

FACULTY OF ENGINEERING AND
TECHNOLOGY
BACHELOR OF TECHNOLOGY

Cloud Computing (303105364)

SEMESTER 6th (3rd YEAR)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Laboratory Manual

PARUL UNIVERSITY

FACULTY OF ENGINEERING & TECHNOLOGY
PARUL INSTITUTE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

STUDENT DETAILS

NAME :
ENROLLMENT NO :
SUBJECT : Cloud Computing
SUBJECT CODE : 303105364
SEMESTER : 6TH
FACULTY NAME : Pathan Bilalkhan R.

FACULTY OF ENGINEERING AND TECHNOLOGY
BACHELOR OF TECHNOLOGY
CERTIFICATE

This is to certify that Mr./Ms _____
with Enrollment no: _____ has Successfully
completed his/her Laboratory Experiments in the Cloud
Computing (303105364) from the Department of Computer
Engineering during the academic year 2025-26

Date of Submission:

Staff In charge:

Head of Department:

INDEX

Sr. No	Experiment Title	Number of Hours	Page No.	Date of Performance	Date of Assessment	Marks (out of 10)	Sign
1	Understanding single core and multi core Architecture.	2					
2	Understanding Computer Network fundamentals and Designing LANs.	2					
3	Implementation of Infrastructure as a service (IaaS) using Hypervisors.	2					
4	Implementation of private cloud platform using open stack cloud.	2					
5	Working with IaaS of Public cloud platforms.	2					
6	Implementation of Platform as a service (PaaS) in private cloud environment.	2					
7	Implementation Platform as a service (PaaS) in public cloud environment.	2					
8	Implementation Software as a service (SaaS) in private cloud environment.	2					
9	Implementing Software as a service (SaaS) in public cloud environment.	2					
10	Implementation of Storage as a service (SaaS).	2					

Practical-1

Aim: Understanding single core and multi core Architecture.

Core:

In computing, a core refers to the processing unit within a CPU that executes instructions from programs. A single core can handle one task (or thread) at a time, while modern CPUs typically have multiple cores to enable parallel processing, improving performance. Each core has its own ALU (Arithmetic Logic Unit) and control unit, allowing independent operations. Multi-core processors are common in devices, supporting multitasking and efficient execution of complex applications. The number of cores directly impacts the ability to run simultaneous processes smoothly.

Types of Cores:

Single-Core Processor: Contains one core to execute tasks sequentially. It performs one operation at a time and is suitable for simple tasks but struggles with multitasking and modern applications.

Multi-Core Processor: Includes two or more cores in a single CPU, enabling parallel processing for improved performance and efficiency in multitasking and demanding applications.

Need of Multi-Core Processor:

Single-core processors are adequate for basic operations like simple browsing or office tasks. Multi-core processors are essential for modern systems that run complex applications, multitasking, gaming, or heavy computations, as they divide workloads across multiple cores for better speed and responsiveness.

Architecture of a Single-Core Processor

Design

1. **Simplicity in Design:** Compared to multi-core processors, designing a single-core processor is simpler since only one core is present. There's no need to implement complex communication mechanisms among multiple cores.
2. **Ease of Software Development:** Developing software for a single-core processing system is easier compared to multi-core systems, as there are no concerns about thread synchronization or resource contention among cores.

Throughput

1. **Sequential Task Execution:** Throughput is lower because tasks are executed sequentially, one after the other.
2. **Limited Parallelism:** While multitasking can reduce CPU idle time, true parallelism cannot be achieved, limiting performance for computationally intensive workloads.

Cost and Power Consumption

1. **Cost Efficiency:** The cost of a single-core system is significantly lower since only one core is used, reducing manufacturing complexity and silicon usage.
2. **Power Efficiency:** Single-core processors consume less power, making them suitable for devices with strict energy constraints.

Applications

1. **Low-Power, Simple Systems:** Despite the dominance of multi-core processors, single-core systems are still used in applications where low power consumption and simplicity are priorities.
2. **Embedded Systems:** Devices such as microwaves, washing machines, and IoT devices, which do not require high computational power, continue to rely on single-core processors.

Architecture of a Multi-Core Processor

Design

1. **Complex Design:** Multi-core processors are more complex than single-core processors due to the need for effective communication mechanisms, such as interconnects, shared buses, or ring architectures, to allow cores to collaborate efficiently.
2. **Software Development Challenges:** Developing software for multi-core processors is challenging as it requires parallel programming techniques, thread management, and synchronization mechanisms to ensure optimal performance and avoid issues like race conditions or deadlocks.

Throughput

1. **High Throughput:** By allowing multiple cores to execute tasks simultaneously, multi-core processors achieve significantly higher throughput compared to single-core processors.
2. **Parallel Task Execution:** Supports true parallelism, enabling better performance for multithreaded applications such as video rendering, gaming, and scientific simulations.

Cost and Power Consumption

1. **Higher Cost:** Multi-core processors are more expensive to manufacture due to the complexity of integrating multiple cores on a single chip and ensuring efficient communication among them.
2. **Increased Power Consumption:** Although more power-efficient per task compared to increasing clock speed, the overall power consumption is higher than single-core processors, especially in high-performance configurations.

Applications

1. **High-Performance Systems:** Multi-core processors are used in systems requiring significant computational power, such as servers, gaming consoles, and workstations.
2. **Parallel Computing:** Applications like machine learning, big data processing, and scientific simulations benefit from the parallel processing capabilities of multi-core processors.
3. **Consumer Electronics:** Modern smartphones, tablets, and laptops use multi-core processors for efficient multitasking and responsiveness.

Physical and Logical Cores:

A multi-core processor contains multiple physical cores that allow parallel processing to improve performance. To further enhance core utilization, Hyper-Threading Technology (HTT) is employed. This technology creates logical cores by enabling a single physical core to handle multiple threads simultaneously. Logical cores help reduce the idle time of the CPU by optimizing how tasks are executed, allowing it to switch between threads efficiently.

This means that each physical core is virtualized as multiple logical cores (commonly two per physical core), enabling better multitasking and improving performance in workloads that benefit from parallel execution, such as video rendering or running virtual machines. However, it is important to note that logical cores share the same physical core's resources (e.g., cache and execution units), so their performance boost depends on the workload.

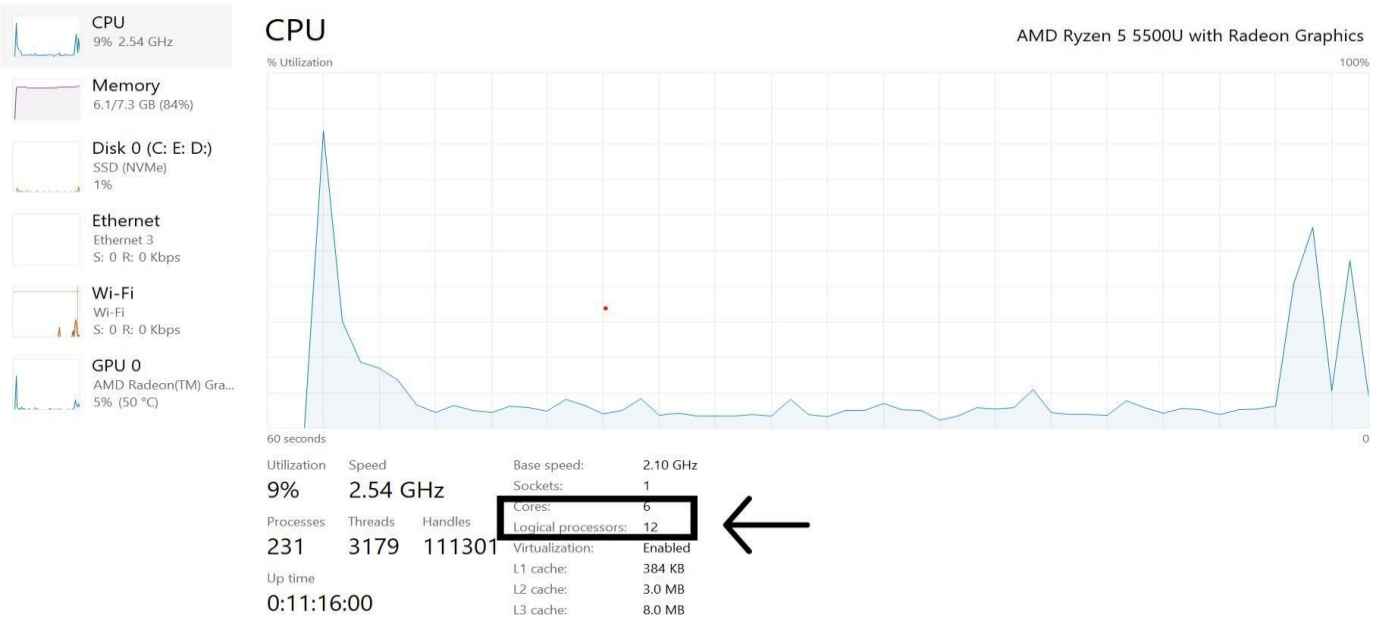
Exploring cores in personal computer:

First way:

1. Go to Task manager
2. Click on the CPU
3. Each core can work with 2 threads simultaneously using Hyper-Threading Technology.
Hence Logical cores will be double of the Physical cores

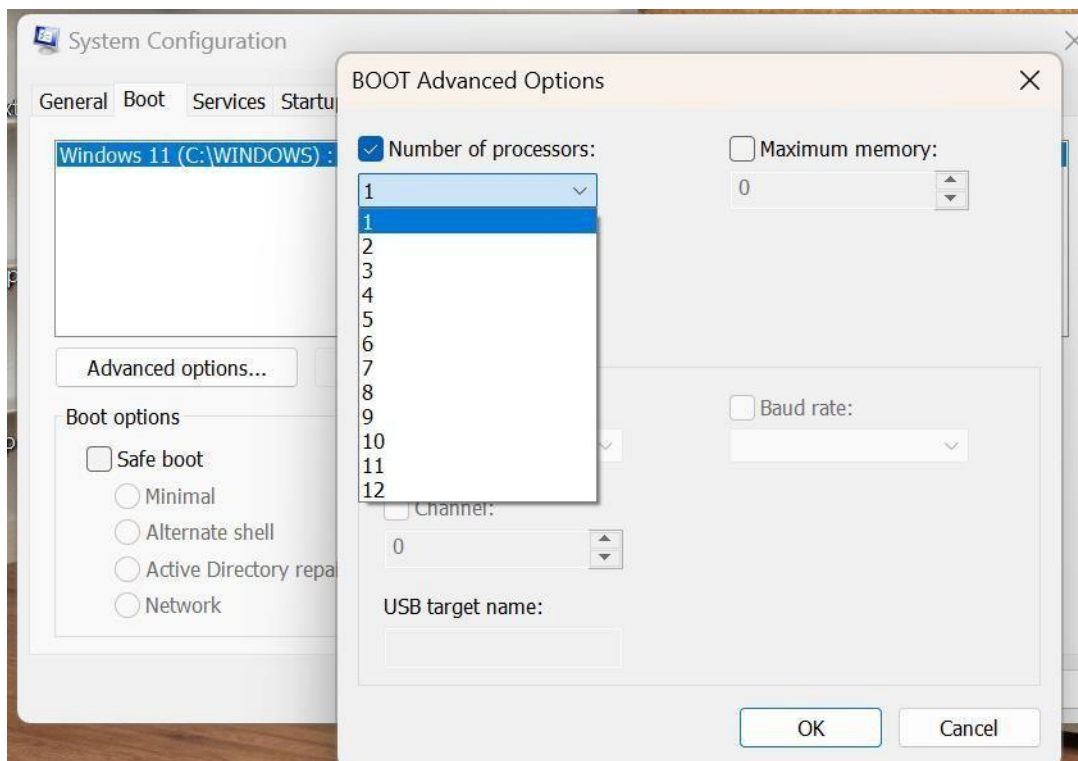
Performance

Run new task ...



Second Way:

1. Open System Configuration:
2. Press Win + R to open the Run dialog.
3. Type msconfig and press Enter.
4. Navigate to the Boot Tab:
5. In the System Configuration window, go to the Boot tab.
6. Open Advanced Options:
7. Under the Boot tab, click on Advanced Options.
8. Check Processor Count:
9. In the Boot Advanced Options window, you'll see a checkbox labelled Number of processors.
10. Check the box, and the drop-down menu below will display the number of cores available in your CPU.
11. Exit Without Changing:
12. If you are just checking the core count, click Cancel to avoid making unintended changes, and then close the System Configuration window.



Comparing Single and Multi-Core Processor:

Feature	Single-Core Processor	Multi-Core Processor
Definition	A processor with one core or processing unit	A processor with multiple cores or processing units
Parallel Processing	Limited to executing one instruction at a time	Can execute multiple instructions simultaneously
Performance	Lower performance for multitasking and complex tasks	Higher performance for multitasking and complex tasks
Power Consumption	Generally lower power consumption	Higher power consumption but more efficient per task
Heat Generation	Generates less heat	Generates more heat due to multiple cores
Efficiency	Less efficient for modern applications	More efficient for modern applications that require parallel processing
Cost	Typically cheaper	Generally more expensive
Suitable For	Basic tasks like web browsing, word processing	Intensive tasks like gaming, video editing, and scientific computations
Example	Early desktop computers	Modern desktops, laptops, and servers

Practical-2

Aim: Understanding Computer Network fundamentals and Designing LANs.

Computer Network:

A computer network is a group of devices that are connected and can share resources and data with each other. Computer networks are made up of nodes and links, which are the basic building blocks of the network. Nodes are devices that send and receive data, such as modems, hubs, and switches. Links are the connections between nodes, such as cable wires or optical fibres.

Network Model:

Sharing data on the network involves many challenges and Network Model is a layered structure which solves every challenge in different layers enabling smooth functioning in the network.

Types:

1. TCP/IP Model
2. OSI Model

TCP/IP Model:

Stands for Transmission Control Protocol/Internet Protocol. This model consists of 4 layers namely,

1. Application Layer
2. Transport Layer
3. Network Layer
4. Network Access Layer

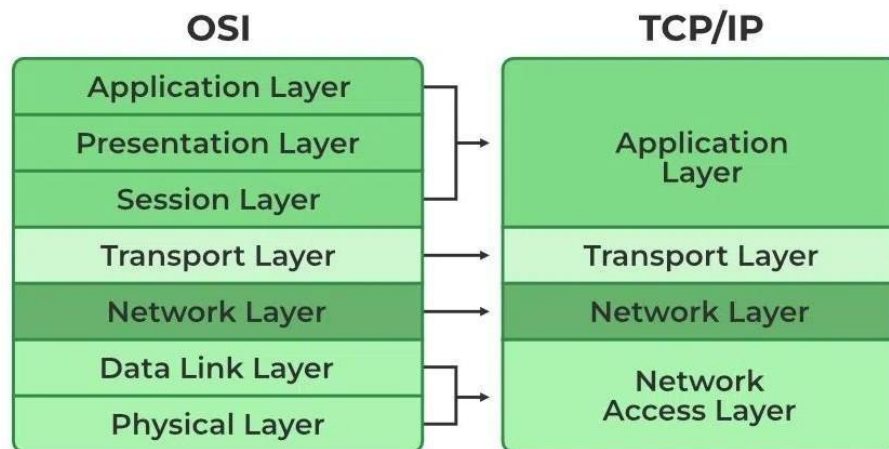
TCP/IP model is one the oldest and most widely used model till date.

OSI Model:

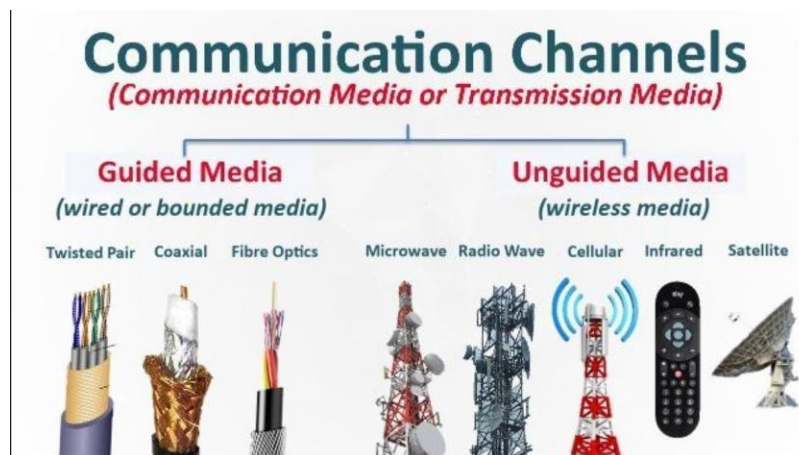
Stands for Open System Interconnection. This model consists of 7 layers namely,

1. Application layer
2. Presentation layer
3. Session layer
4. Transport layer
5. Network layer
6. Data link layer
7. Physical layer.

Each layer gives services to the layer just below it and take services from layer above it.



Transmission Media:



Transmission media talks about how data is exactly transferred. It can be wired in other words known as **Guided Media** as well. **Unguided Media** involves data transfer based on electromagnetic waves.

Twisted Pair Cable: To reduce electromagnetic inference two insulated copper wires are twisted together. Generally used in telephone systems, Ethernet networks.

Coaxial Cable: Broadband Internet, TV cable are examples of this coaxial cable. This cable contains single copper wire with multiple layers of protection. This is expensive than Twisted Pair Cable.

Fibre Optics: Works on the principle of total internal reflection. This is made of sand which looks very similar to glass. Uses light signals to transmit data.

Unguided Media: This method involves data transfer without the need of cables. But this method involves various security issues. Radio, Cellular Network works on Unguided Media.

Networking Devices: These are hardware devices in the computer networks which ensures smooth communication over the internet. Various devices work at various levels of network model.

Active Devices: Routers, Switches

Passive Devices: Hubs, Repeaters

End Devices: Servers, Workstations

Routers:



This device works at network layer. Router connects LAN to WAN by providing access to the internet with the help of Ip address.

Switches:



Mainly used in Data Link Layer but sometimes used in network layer as well. Switch connect various devices within same network. Based on MAC address switch forwards data to target machine only.

Network Topologies:

- Star
- Ring
- Bus
- Mesh

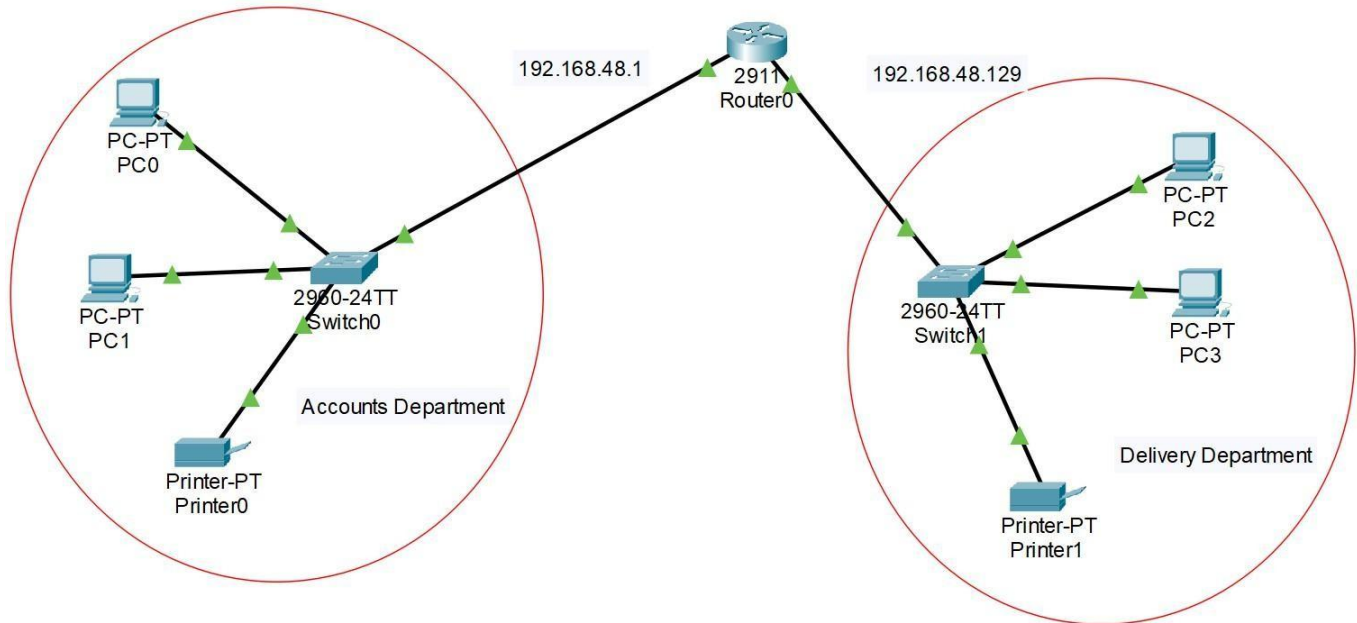
Network Protocols:

- TCP
- UDP

Designing a Network:

Create a network with 2 departments. Use appropriate number of switches and routers.

Use the network addresses 192.168.40.0. use appropriate cables to connect all the devices.



Output:

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC0	Printer1	ICMP		0.000	N	0	(edit)	(delete)
	Successful	PC1	PC2	ICMP		0.000	N	1	(edit)	(delete)
	Successful	Printer0	Printer1	ICMP		0.000	N	2	(edit)	(delete)

Practical-3

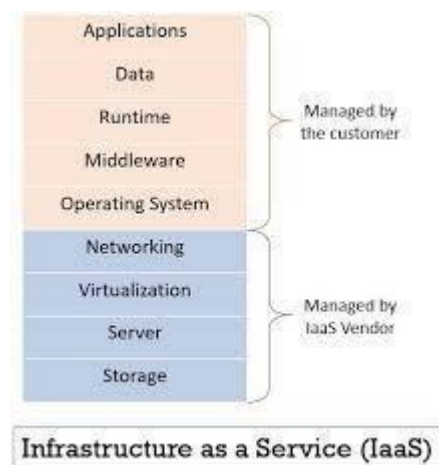
Aim: Implementation of Infrastructure as a service (IaaS) using Hypervisors.

IaaS: Before getting into the practical let us try to understand about infrastructure as a service.

For Hosting a software application, we need lot of hardware in the backend. Hardware includes server, storage, networking etc.

This becomes costly and headache for developers to always look after the working of this underlaying hardware in addition to maintaining their software application.

IaaS solves this problem by providing virtualized resources via internet. All we need to good internet connection. We can get server, storage, networking etc without setting up on our own. IaaS commits that cloud provider will be solely responsible for managing underlaying



infrastructure. But operating system and runtime environments should be seen by user itself.

Hypervisor: Hypervisors are the implementation of vitalization concept. This is the backbone of IaaS, because only Hypervisors enable resource sharing and dynamic allocation of virtualized environments. Hypervisor allows multiple operating systems and virtual machines to run on the same physical hardware by abstracting and allocating resources such as CPU, Memory, Storage and Network.

Types of Hypervisors:

Type 1 Hypervisor (Bare-Metal Hypervisors):

These run directly on the physical hardware without requiring underlying OS.

Example: KVM (kernel virtual machine) which we are going to implement now and other hypervisors also exist like VMware ESXi, Microsoft Hyper-V etc.

Type-2 Hypervisors (Hosted Hypervisors):

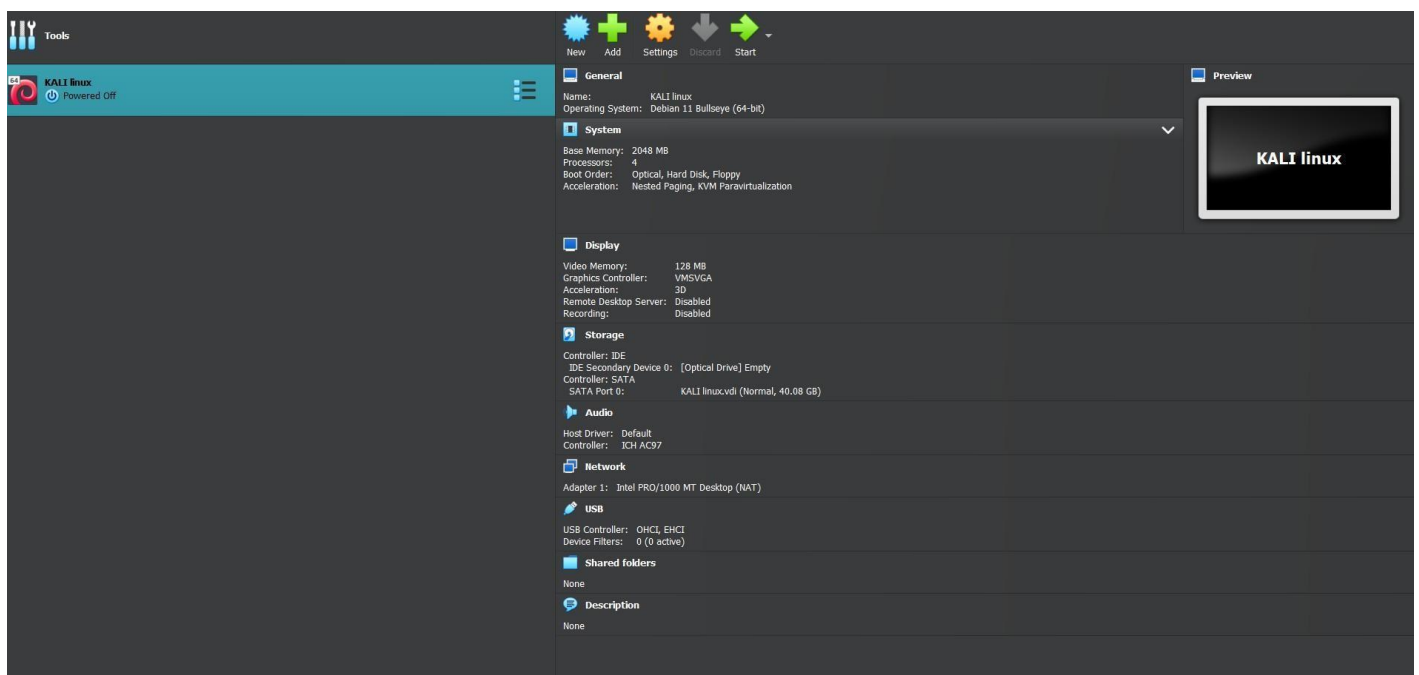
These run on top of an existing operating system and relay on the host to manage hardware resources. This will be useful for development, testing and for small scale virtualized environments.

Example: VMware workstation, VirtualBox.

Let us create a virtual machine using KVM:

Things to keep in mind:

1. We are on windows operating system, and we have installed virtual box, on top of virtual box we had Linux system.
2. Using Linux machine present in the virtual box we need to create virtual machine with the help of KVM and virtual manager.



Use lscpu command:

In this section, we have virtualization features. It should be showing full. And hypervisor type will be KVM (kernel virtual machine)

```
jagadeep@kali: ~
$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:    0-3
Vendor ID:              GenuineIntel
Model name:             11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz
CPU family:             6
Model:                 141
Thread(s) per core:     1
Core(s) per socket:     4
Socket(s):              1
Stepping:               1
BogoMIPS:               4600.00
Flags:                  fpu vme de pse mtr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx rdtscp lm constant_tsc rep_good nopl xtopology nonstop_tsc cpuid tsc_known_freq pni pclmulqdq sss
e3 cx16 pcid sse4_1 sse4_2 movbe popcnt aes rdrand hypervisor lahf_lm abm 3dnowprefetch ibrs_enhanced fsgsbase bmi1 bmi2 invpcid rdseed clflushopt arat md_clear flush_l1d arch_capabilities

Virtualization features:
Hypervisor vendor:      KVM
Virtualization type:    full
Caches (sum of all):
  L1d:                   192 KiB (4 instances)
  L1i:                   128 KiB (4 instances)
  L2:                     5 MiB (4 instances)
  L3:                     96 MiB (4 instances)
NUMA:
  NUMA node(s):          1
  NUMA node0 CPU(s):     0-3
Vulnerabilities:
Gather data sampling:    Not affected
Itlb multihit:           Not affected
L1tf:                    Not affected
Mds:                     Not affected
Meltdown:                Not affected
Mmio stale data:         Not affected
Reg file data sampling:  Not affected
Retbleed:                Mitigation; Enhanced IBRS
Spec rstack overflow:    Not affected
Spec store bypass:       Vulnerable
Spectre v1:              Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:              Mitigation; Enhanced / Automatic IBRS; RSB filling; PRRS0-eIBRS SW sequence; BHI SW loop, KVM SW loop
Srbds:                   Not affected
Tsx async abort:         Not affected
```

Now use the following command:

```
sudo apt update && sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients
bridge-utils virt-manager
```

qemu-kvm

- QEMU is an open-source emulator that allows running virtual machines.
- KVM (Kernel-based Virtual Machine) is a Linux-based hypervisor for running VMs efficiently.

libvirt-daemon-system

- Libvirt is a toolkit for managing virtual machines.
- libvirt-daemon-system provides system-wide services for managing VMs.

libvirt-clients

- It contains command-line tools like virsh to interact with virtual machines.

bridge-utils

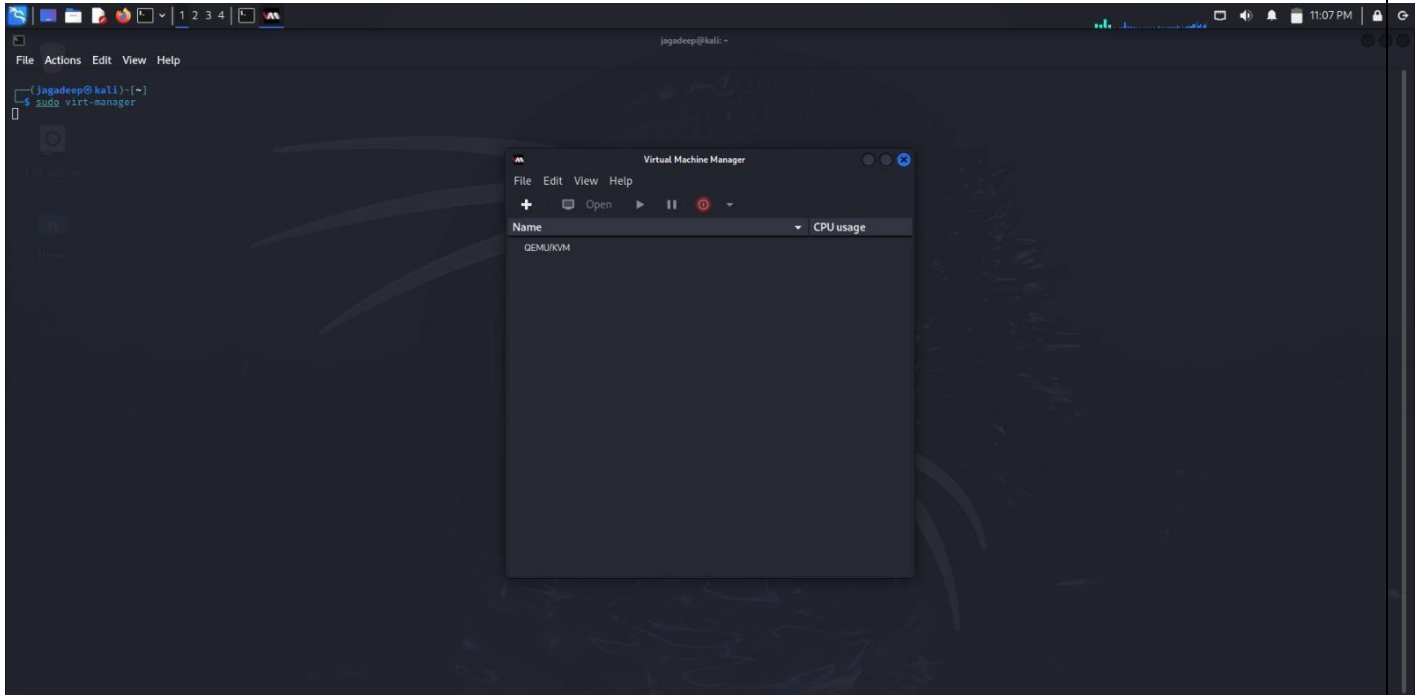
- It provides network bridging capabilities.
- Used for connecting virtual machines to the internet or the host network.

virt-manager

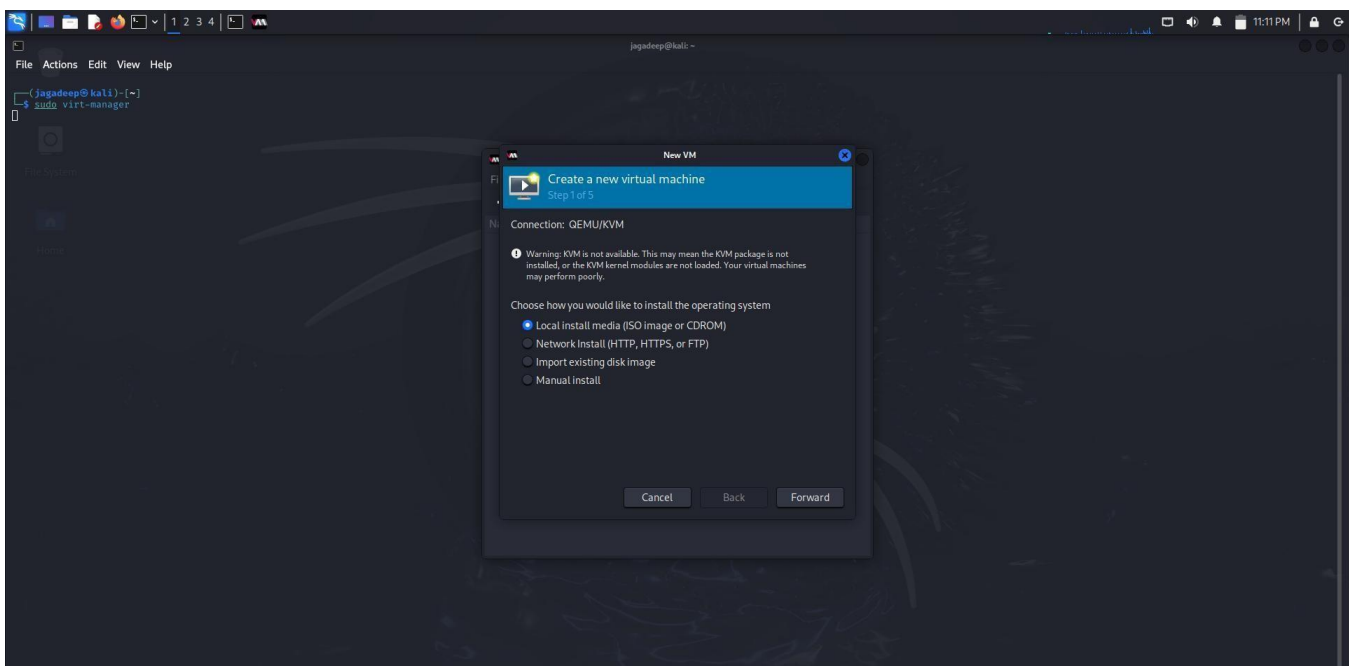
- Virt-Manager (Virtual Machine Manager) is a GUI tool for managing VMs.

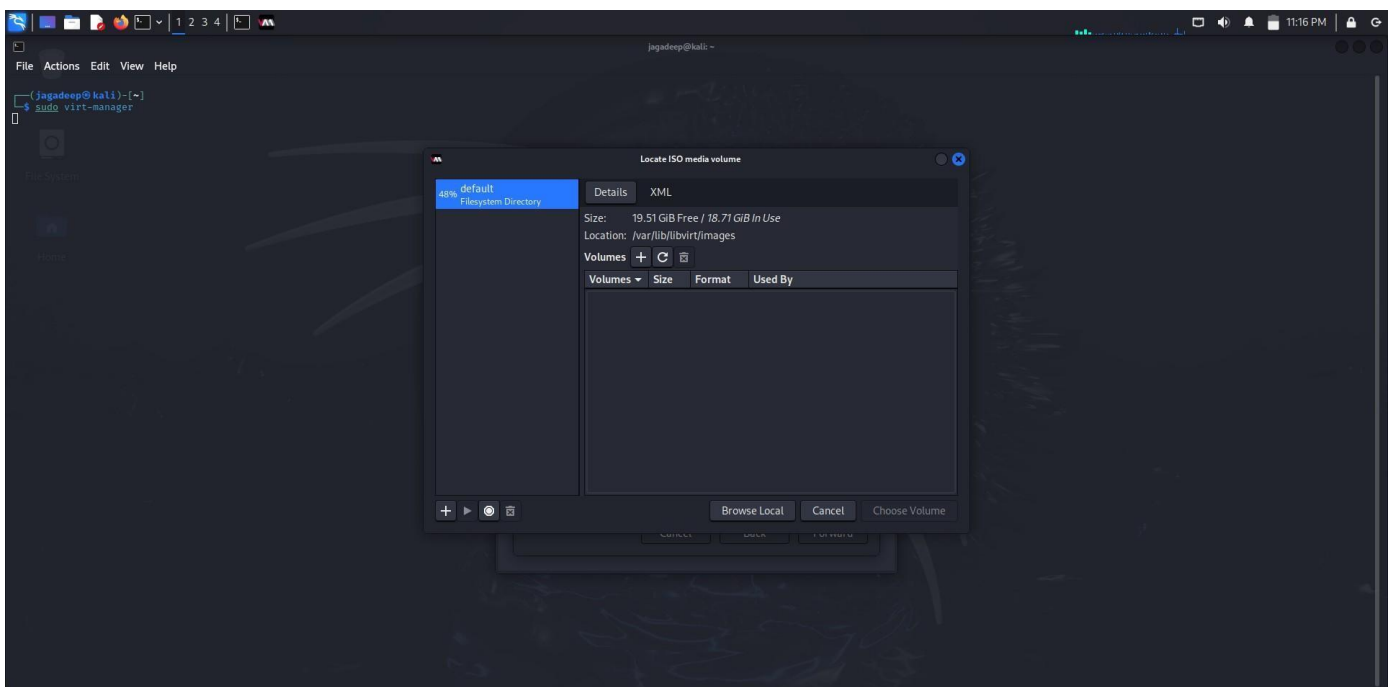
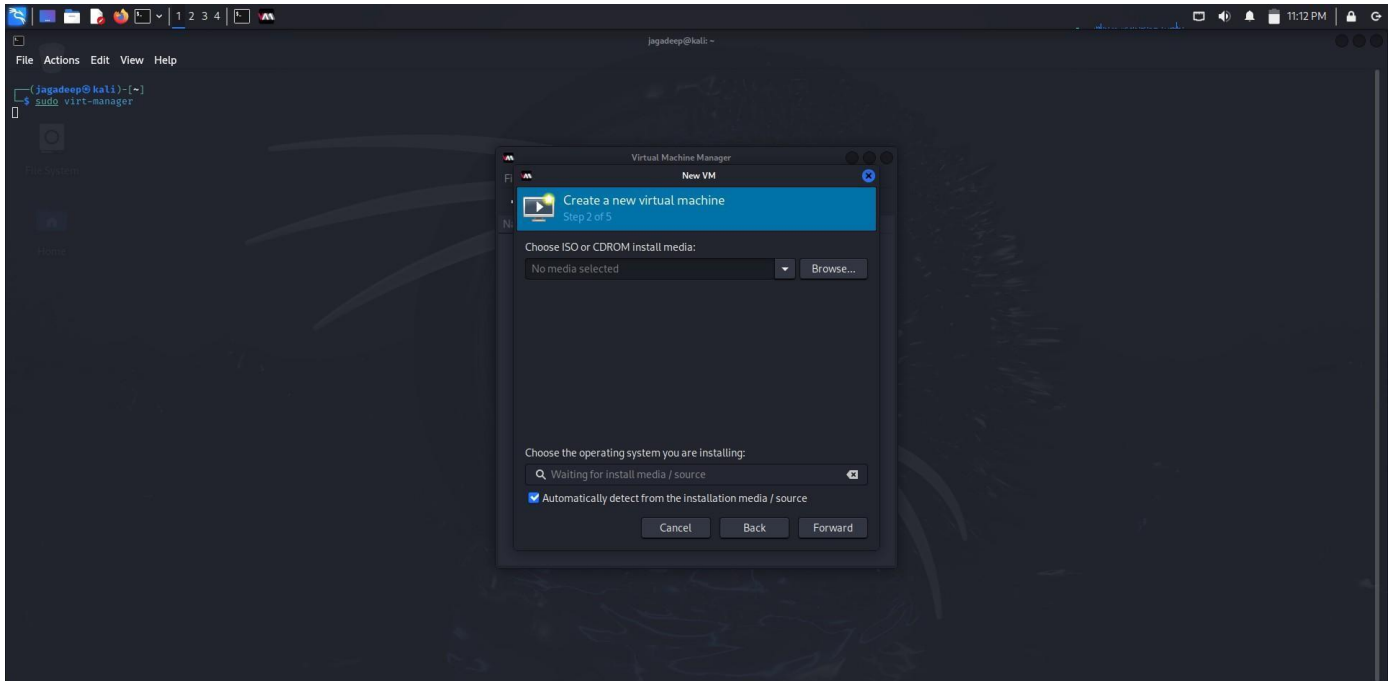
- It provides an easy-to-use interface to create, delete, and manage virtual machines.

Now sudo virt-manager should open up an GUI for creating VM's



Now click on file, cerate new VM. And follow the further steps in setting up the vm. We will be asked to select iso file as well.

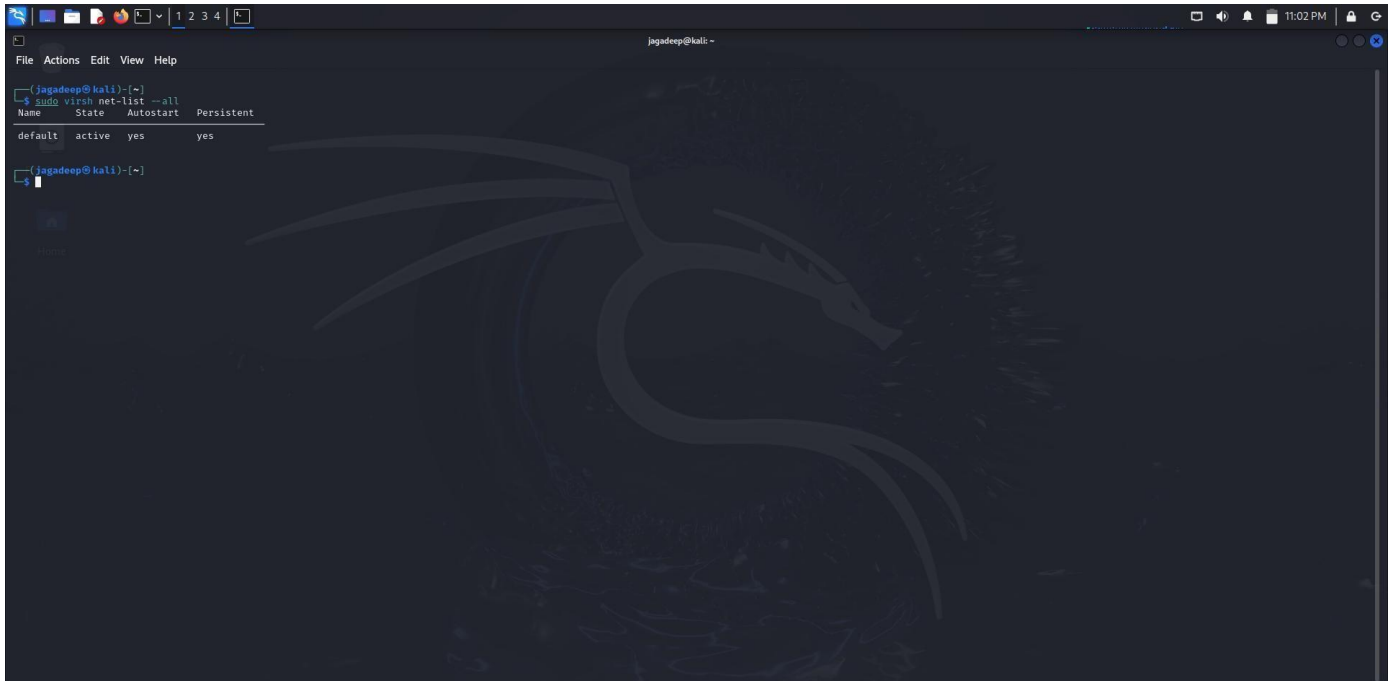




After setting up the virtual machine we can confirm it by using the following command

`Sudo virsh net-list --all`

This will list all the virtual machine running. We can confirm the status of the created virtual machine.



To enter the created virtual machine, there are various options available

Method	Command
GUI (virt-manager)	<code>virt-manager</code> → Click VM
Terminal Console	<code>virsh console ubuntu-vm1</code>
SSH (if configured)	<code>ssh user@vm-ip</code>
VNC (for remote GUI)	<code>virsh vncdisplay ubuntu-vm1</code>

Conclusion:

P

Practical-4 Aim: Working with IaaS of Public

cloud platforms Google Cloud:

In the following manual we are going to work Google Cloud Platform. GCP is now providing more than 200 services for its users. Let us take an overview before heading into practical.

Google Cloud IaaS (Infrastructure as a Service) provides scalable computing, storage, and networking resources on demand.

Compute Engine is its core IaaS service, offering customizable virtual machines (VMs) for various workloads.

Persistent Disk and Local SSDs provide high-performance storage options for VMs.

Cloud Load Balancing distributes traffic across multiple instances for reliability and performance.

VPC (Virtual Private Cloud) allows secure networking with fine-grained control over IP ranges and firewall rules.

Preemptible VMs and Spot VMs offer cost-effective options for non-critical workloads.

Autoscaling dynamically adjusts the number of VMs based on demand.

Identity and Access Management (IAM) ensures secure access control for cloud resources.

Cloud Storage integrates with IaaS for scalable object storage.

Compute Engine integrates with Kubernetes (GKE) and AI/ML services for advanced workloads.

IAAS Offering by GCP:

GCP offers several Infrastructure as a Service (IaaS) option, including Compute Engine, Cloud Storage, Cloud SQL, Cloud Spanner, and Google Kubernetes Engine (GKE).

Now Let us get familiar with Compute Engine Service in GCP.

Google Compute Engine:

Google Cloud **Compute Engine** is an **Infrastructure as a Service (IaaS)** offering that provides scalable, flexible, and customizable **virtual machines (VMs)** to run workloads in the cloud. Here are key aspects of Compute Engine:

1. Virtual Machines (VMs)

- Offers different machine types:

Standard, High-Memory, High-CPU, and GPU-based VMs.

- Supports custom VM configurations with specific CPU and RAM requirements.

2. Persistent Storage Options

- Persistent Disks (Standard & SSD): Durable and high-performance storage.
- Local SSDs: Ultra-fast, temporary storage attached to VMs.
- File store: Managed file storage for applications requiring shared file access.

3. Networking Features

- Virtual Private Cloud (VPC): Custom networks, firewall rules, and subnets.
- Load Balancing: Distributes traffic across multiple VM instances.
- Cloud NAT: Provides outbound internet access without exposing internal VMs.

4. Preemptible and Spot VMs

- Preemptible VMs: Short-lived, cost-effective instances (can be terminated anytime).
- Spot VMs: Even cheaper and suitable for batch jobs or fault-tolerant applications.

5. Autoscaling & Instance Groups

- Managed Instance Groups (MIGs): Automatically scales VM instances based on demand.
- Unmanaged Instance Groups: Manually managed set of VM instances.

6. Security & Identity Management

- IAM (Identity and Access Management) controls access to VMs.
- Shielded VMs provide advanced protection against rootkits and malware.

7. Operating System Support

- Supports Linux and Windows Server operating systems.
- Custom OS images can be uploaded and used in VMs.

8. Integration with Other GCP Services

- Works seamlessly with Cloud Storage, BigQuery, Kubernetes (GKE), and AI/ML services.

9. Cost Management

- Offers Sustained Use Discounts (automatic cost reduction for long-running VMs).
- Committed Use Contracts for predictable workloads with further cost savings.

10. Machine Learning & AI Acceleration

- Supports GPUs and TPUs for AI/ML workloads and high-performance computing (HPC)

Using Compute Engine Service:

First step is to Create a project. I has Kept name as ExploringIAAS.

Google Cloud

Search (/) for resources, docs, products, and more

Search

New Project

You have 20 projects remaining in your quota. Request an increase or delete projects. [Learn more](#)

[Manage Quotas](#)

Project name *
ExploringIAAS

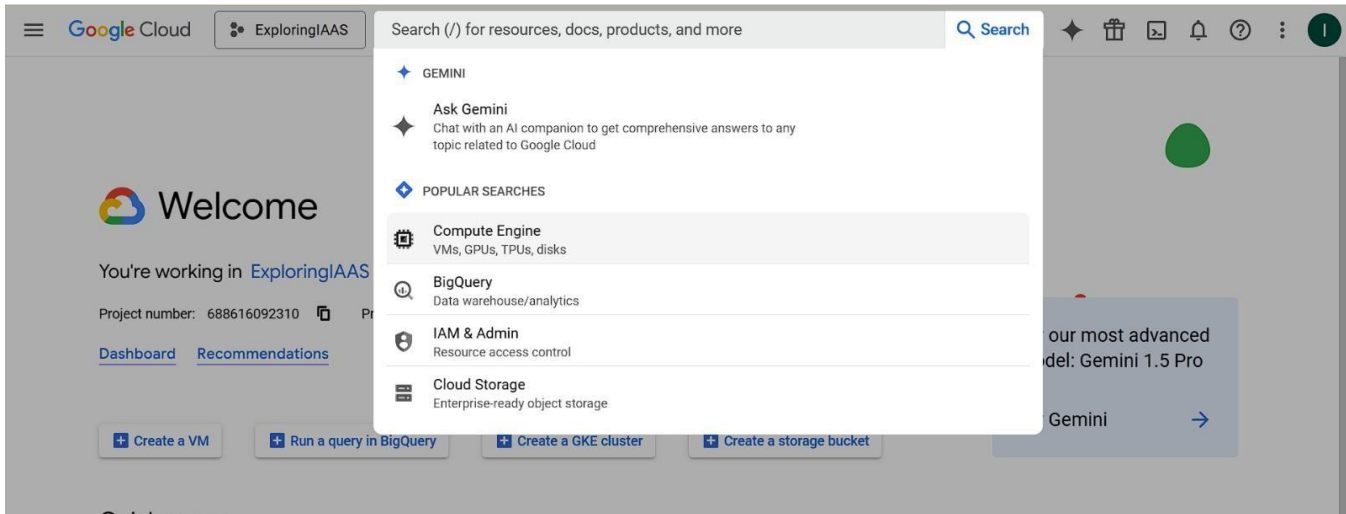
Project ID: exploringiaas. It cannot be changed later. [Edit](#)

Location *
No organization [Browse](#)

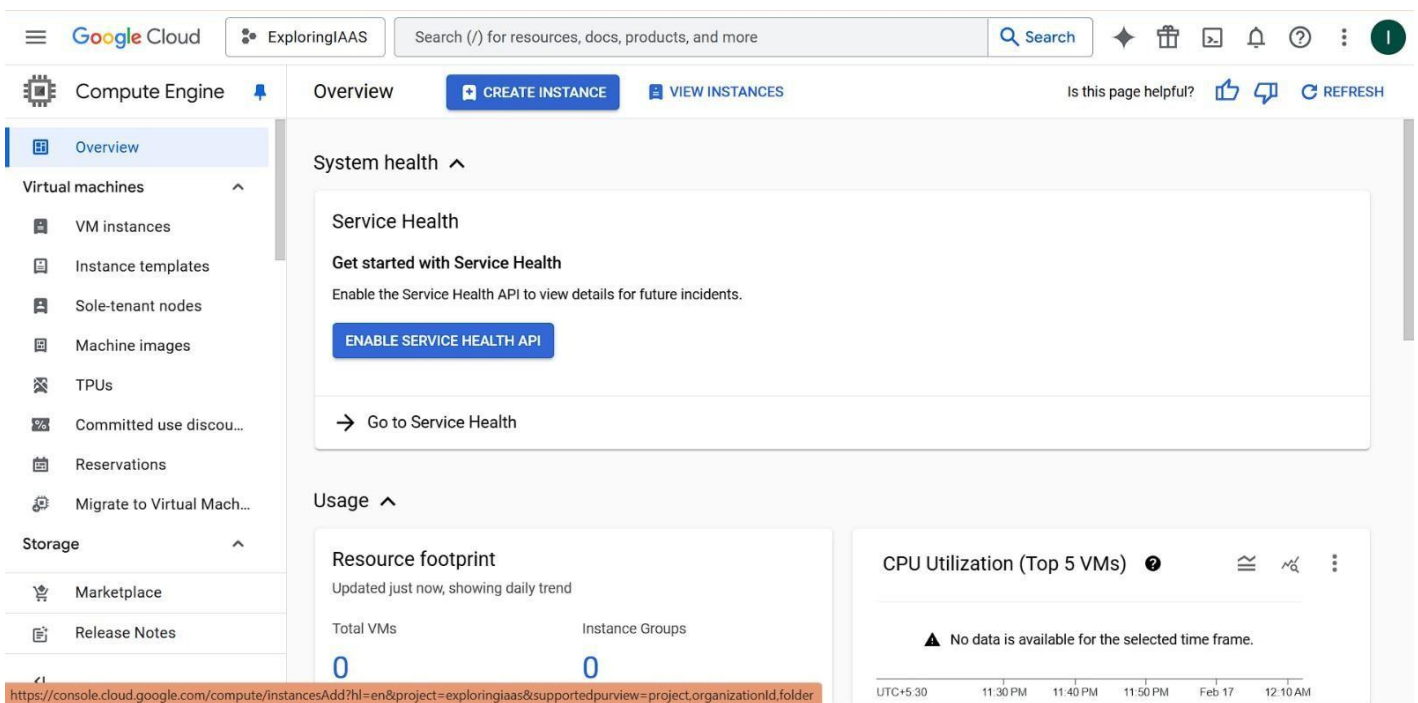
Parent organization or folder

Create Cancel

2nd step is to select compute engine service from the search box.



Now 3rd step is to click on create instance option in the compute engine page.



5th step is to add configurations to our virtual machine. This step includes selecting region, selecting virtual machine types, number of CPU cores, and selecting operating system.

Google Cloud

Exploring IaaS

Search (/) for resources, docs, products, and more

Search

Create an instance

CREATE VM FROM...

EQUIVALENT CODE

Machine configuration

e2-medium, us-central1

OS and storage

Debian GNU/Linux 12 (bookworm)

Networking

1 network interface

Observability

Security

Advanced

Machine configuration

Name *

firstvmmachine

Region *

us-central1 (Iowa)

Zone *

Any

Region is permanent

Google will choose a zone on your behalf, maximizing VM obtainability. Zone is permanent.

General purpose

Compute optimized

Memory optimized

Storage optimized

GPUs

Machine types for common workloads, optimized for cost and flexibility

Series	Description	vCPUs	Memory	CPU Platform
☐ C4	Consistently high performance	2 - 192	4 - 1,488 GB	Intel Emerald Rapids
☐ C4A	Arm-based consistently high performance	1 - 72	2 - 576 GB	Google Axion
☐ N4	Flexible & cost-optimized	2 - 80	4 - 640 GB	Intel Emerald Rapids
☐ C3	Consistently high performance	4 - 192	8 - 1,536 GB	Intel Sapphire Rapids
☐ C3D	Consistently high performance	4 - 360	8 - 2,880 GB	AMD Genoa
☒ E2	Low cost, day-to-day computing	0.25 - 32	1 - 128 GB	Intel Broadwell

Monthly estimate

\$25.46

That's about \$0.03 hourly

Pay for what you use: no upfront costs and per second billing

Item	Month
2 vCPU + 4 GB memory	
10 GB balanced persistent disk	
Total	

[Compute Engine pricing](#)
[LESS](#)

CREATE

CANCEL

EQUIVALENT CODE

Next step is to select the boot disk configurations.

Storage

Boot disk

Name	firstvmmachine
Image	debian-12-bookworm-v20250212
Size	10 GB
Interface type	SCSI
Type	Balanced persistent disk
Encryption type	Google-managed
Mode	Boot, read/write
Snapshot schedule	None

Deletion rule

When deleting instance

☐ Keep disk
☒ Delete disk

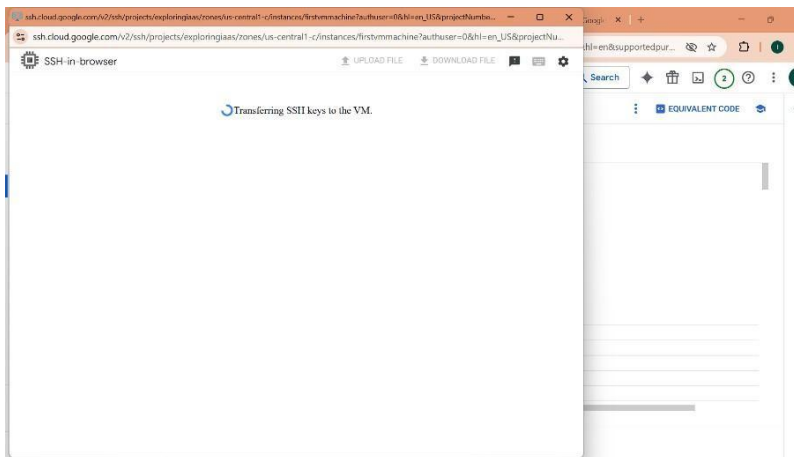
Device name

Used to reference the device for mounting or resizing.

Device name

SAVE
CANCEL

Now we can access the virtual machine using SSH options. After providing Authorization we can use the virtual machine. We can add workloads to the machine.



We can install Apache webserver through the terminal and host applications on the virtual machine as well. It is very important to turn off the virtual machine if it is no longer being used.

INSTANCES							
OBSERVABILITY							
INSTANCE SCHEDULES							
VM instances							
Filter Enter property name or value							
Status	Name	Zone	Recommendations	In use by	Internal IP	External IP	Connect
☐	firstvmmachine	us-central1-c			10.128.0.2 (nic0)		SSH
This instance is stopped							
HIDE							

ssh.cloud.google.com/v2/ssh/projects/exploringiaas/zones/us-central1-c/instances/firstvmmachine?authuser=0&hl=en_US&projectNumber=688616092310&useAdminProxy=true&pageViewl...



SSH-in-browser

UPLOAD FILE

DOWNLOAD FILE



Linux firstvmmachine 6.1.0-31-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.128-1 (2025-02-07) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

immadisettivarun@firstvmmachine:~\$ ls

immadisettivarun@firstvmmachine:~\$ mkdir webproject

immadisettivarun@firstvmmachine:~\$ ls

webproject

immadisettivarun@firstvmmachine:~\$ cd webproject

immadisettivarun@firstvmmachine:~/webproject\$ cat index.html

cat: index.html: No such file or directory

immadisettivarun@firstvmmachine:~/webproject\$ touch index.html

immadisettivarun@firstvmmachine:~/webproject\$ cat>index.html

<doctype html>

<html>

<body>

<h1>hello world</>

</body>

</html>

^C

immadisettivarun@firstvmmachine:~/webproject\$

Conclusion:

Practical-6

Aim: Implementation of PaaS in Private Cloud Environment

Introduction:

Platform as a Service (PaaS) is a cloud computing model that provides ready-to-use application development and deployment platforms. In a private cloud, PaaS offers better control, data security, and compliance for sensitive applications.

Architecture of Private PaaS:

The Private PaaS architecture consists of the following layers:

- **User Layer** – Developers deploy applications
- **Application Layer** – Runtime, middleware, application tools
- **Platform Layer** – Docker containers, orchestration tools
- **Infrastructure Layer** – Virtual machines and storage

Method 1: Using Docker Desktop (Easiest & Recommended for macOS) Step 1: Install Docker Desktop

Go to: <https://www.docker.com/products/docker-desktop>

Download **Docker Desktop for Mac (Apple Silicon / Intel – choose correctly)**

Install and **start Docker Desktop**

Open Terminal and verify:

```
docker --version
```

```
docker-compose --version
```

Step 2: Create PaaS Platform Directory

```
mkdir
```

```
paas-
```

```
platform
```

```
cd paas-
```

```
platform
```

Step 3: Create Docker Compose File

nano docker-compose.yml Paste this content:

```
version: '3'
```

```
services:
```

```
  web:
```

```
    image:
```

```
    nginx
```

```
    ports:
```

```
      - "8080:80"
```

Step 4: Deploy the PaaS Platform

```
docker-compose up -d
```

Step 5: Access Platform

Open browser
and visit:
`http://localhost:8080`

Step 6: User Perspective (App Deployment Example)

```
Create sample app
directory: mkdir
app
cd app
nano index.html
```

Step 7: Modify docker-compose.yml

```
version:
'3'
services:
  web:
    image:
    nginx
    ports:
      - "8080:80"
    volumes:
      -
      ./app:/usr/share/nginx/html
Restart platform:
docker-
compose
down
docker-
compose
up -d
```

Step 8: Stop Platform

```
docker-compose down
```

Commands Summary

Purpose	Command
Check Docker	<code>docker --version</code>
Create directory	<code>mkdir paas-platform</code>
Start platform	<code>docker-compose up -d</code>
Check containers	<code>docker ps</code>
Stop platform	<code>docker-compose down</code>

paas-platform --zsh --94x53

Last login: Wed Jan 28 13:16:32 on ttys000

shubham@Shubhams-MacBook-Air ~ % docker --version

zsh: command not found: docker

shubham@Shubhams-MacBook-Air ~ % docker --version

Docker version 29.1.3, build f52814d

shubham@Shubhams-MacBook-Air ~ % docker --version

docker compose version

Docker version 29.1.3, build f52814d

Docker Compose version v5.0.1

shubham@Shubhams-MacBook-Air ~ % Docker version XX.X.X

Docker Compose version vX.X.X

docker: 'docker version' accepts no arguments

Usage: docker version [OPTIONS]

Run 'docker version --help' for more information

docker: unknown command: docker Compose

Run 'docker --help' for more information

shubham@Shubhams-MacBook-Air ~ % docker compose

Usage: docker compose [OPTIONS] COMMAND

Define and run multi-container applications with Docker

Options:

--all-resources	Include all resources, even those not used by services
--ansi string	Control when to print ANSI control characters ("never" "always" "auto") (default "auto")
--compatibility	Run compose in backward compatibility mode
--dry-run	Execute command in dry run mode
--env-file stringArray	Specify an alternate environment file
-f, --file stringArray	Compose configuration files
--parallel int	Control max parallelism, -1 for unlimited (default -1)
--profile stringArray	Specify a profile to enable
--progress string	Set type of progress output (auto, tty, plain, json, quiet)
--project-directory string	Specify an alternate working directory (default: the path of the, first specified, Compose file)
-p, --project-name string	Project name

Management Commands:

bridge Convert compose files into another model

Commands:

attach	Attach local standard input, output, and error streams to a service's running container
build	Build or rebuild services
commit	Create a new image from a service container's changes
config	Parse, resolve and render compose file in canonical format

paas-platform --zsh --94x53

Commands:

attach	Attach local standard input, output, and error streams to a service's running container
build	Build or rebuild services
commit	Create a new image from a service container's changes
config	Parse, resolve and render compose file in canonical format
cp	Copy files/folders between a service container and the local filesystem
create	Creates containers for a service
down	Stop and remove containers, networks
events	Receive real time events from containers
exec	Execute a command in a running container
export	Export a service container's filesystem as a tar archive
images	List images used by the created containers
kill	Force stop service containers
logs	View output from containers
ls	List running compose projects
pause	Pause services
port	Print the public port for a port binding
ps	List containers
publish	Publish compose application
pull	Pull service images
push	Push service images
restart	Restart service containers
rm	Removes stopped service containers
run	Run a one-off command on a service
scale	Scale services
start	Start services
stats	Display a live stream of container(s) resource usage statistics
stop	Stop services
top	Display the running processes
unpause	Unpause services
up	Create and start containers
version	Show the Docker Compose version information
volumes	List volumes
wait	Block until containers of all (or specified) services stop.
watch	Watch build context for service and rebuild/refresh containers when files are updated

Run 'docker compose COMMAND --help' for more information on a command.

shubham@Shubhams-MacBook-Air ~ % docker run hello-world

Unable to find image 'hello-world:latest' locally

latest: Pulling from library/hello-world

198f93fd5094: Pull complete

Digest: sha256:05813aedc15fb7b4d732e1be879d3252c1c9c25d885824f6295cab4538cb85cd

Status: Downloaded newer image for hello-world:latest

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(arm64v8)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

```
shubham@Shubhams-MacBook-Air ~ % mkdir paas-platform
cd paas-platform
nano docker-compose.yml
```

```
shubham@Shubhams-MacBook-Air paas-platform % nano docker-compose.yml
```

```
shubham@Shubhams-MacBook-Air paas-platform % docker-compose up -d
WARN[0000] /Users/shubham/paas-platform/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 6/8
[+] up 10/10nx [#####] 61.02MB / 61.18MB Pulling
```

```
27.7s
✓ Image nginx Pulled

27.7s
✓ d637807aba98 Pull complete

18.2s
✓ 15a5b76537aa Pull complete

23.0s
✓ f43c327f6738 Pull complete

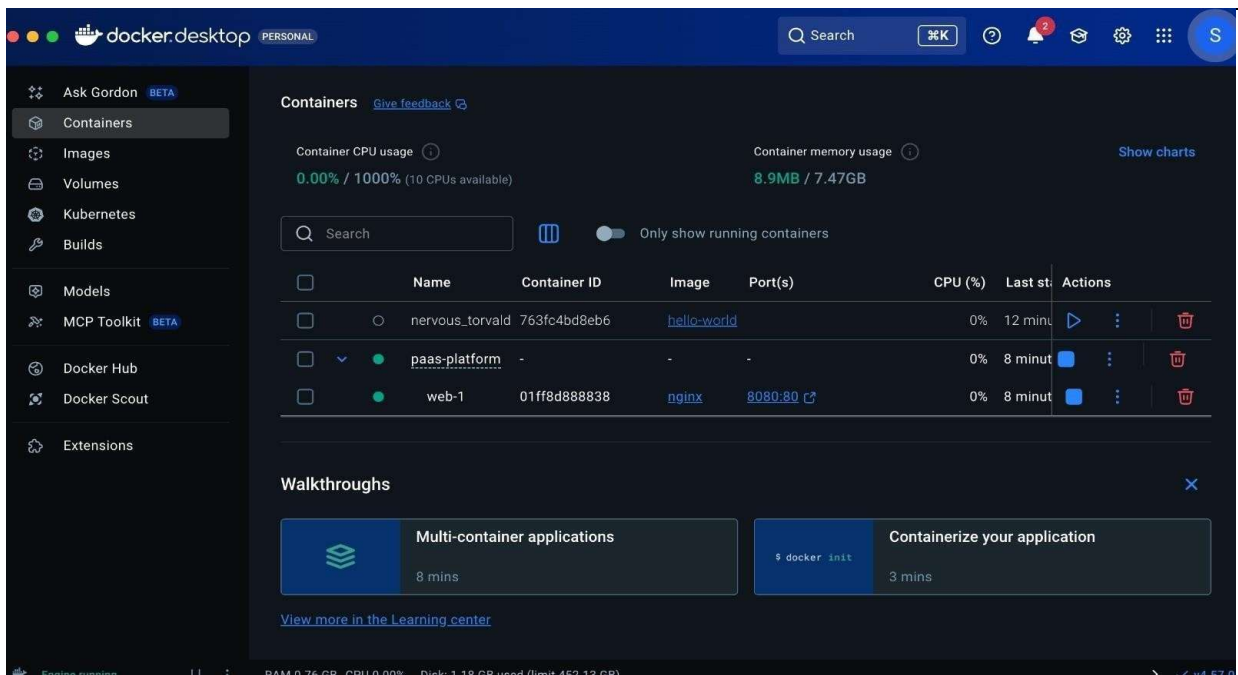
23.0s
✓ 1d5a66120144 Pull complete

23.0s
✓ 96b249cf17a2 Pull complete

23.1s
✓ a178f616fffa Pull complete

23.1s
0.0s
✓ Container paas-platform-web-1 Created

0.1s
shubham@Shubhams-MacBook-Air paas-platform %
```



Conclusion:-

Practical-7

Aim:- Implementation of Platform as a Service (PaaS) in Public Cloud Environment (Railway Cloud)

1. Aim of the Practical

The aim of this practical is to implement Platform as a Service (PaaS) in a public cloud environment using **Railway Cloud**, enabling application deployment without managing underlying infrastructure.

2. Introduction

Platform as a Service (PaaS) provides a cloud-based platform for developing, testing, deploying, and managing applications. In a public cloud environment, PaaS providers handle infrastructure, runtime, and scaling automatically. **Railway Cloud** is a modern PaaS platform that allows developers to deploy applications easily using GitHub integration and automated builds.

3. Architecture of Public PaaS

Public PaaS architecture includes:

- **User Layer** – Application developers
- **PaaS Layer** – Runtime environment, frameworks, APIs
- **Management Layer** – Auto-deployment, monitoring, scaling
- **Infrastructure Layer** – Servers, storage, and networking

4. Requirements

- Laptop/Desktop with internet
- GitHub account

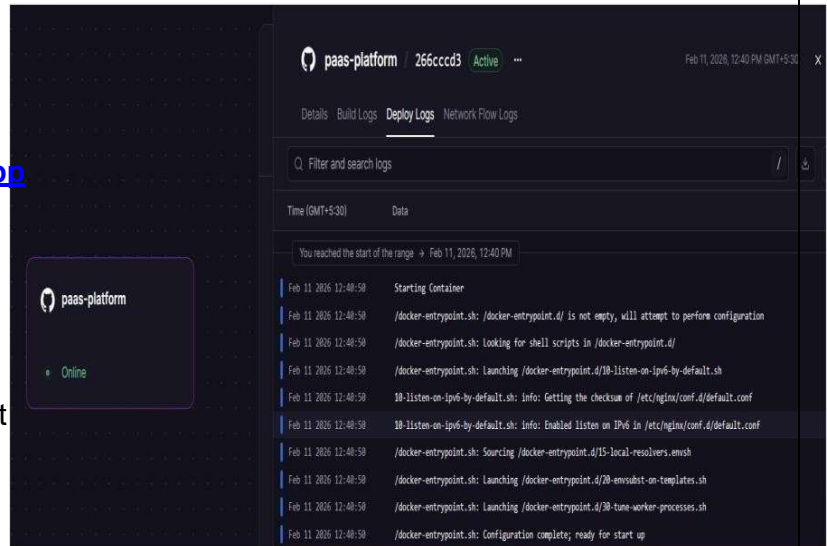
Railway Cloud account (Free Tier)

5. Step-by-Step Implementation using

Railway Cloud Step 1: Create Railway

Cloud Account

1. Visit <https://railway.app>
2. Click **Login**
3. Sign in using **GitHub account**
4. Complete account setup



Step 2: Create New Project

1. Click **New Project**
2. SELECT DEPLOY FROM GITHUB REPO
3. Choose a repository or create a new one
4. Click **Deploy**

Step 3: Create Sample Application

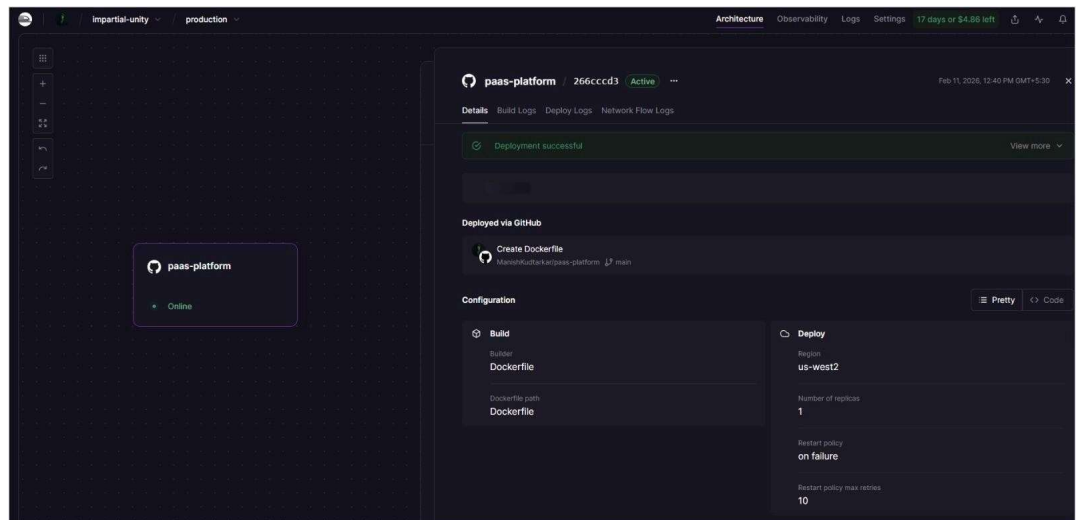
```
Create a Python file app.py:
print("Hello from Public PaaS using Railway
Cloud") Create requirements.txt:
flask
Create main.py (Flask app):
from flask
import Flask
app = Flask(_
name_)

@app.route("/")
def
home():
    return "Hello from Public PaaS using Railway Cloud"

if __name__ == "__main__":
    app.run(host="0.0.0.0",
    port=8080)
```

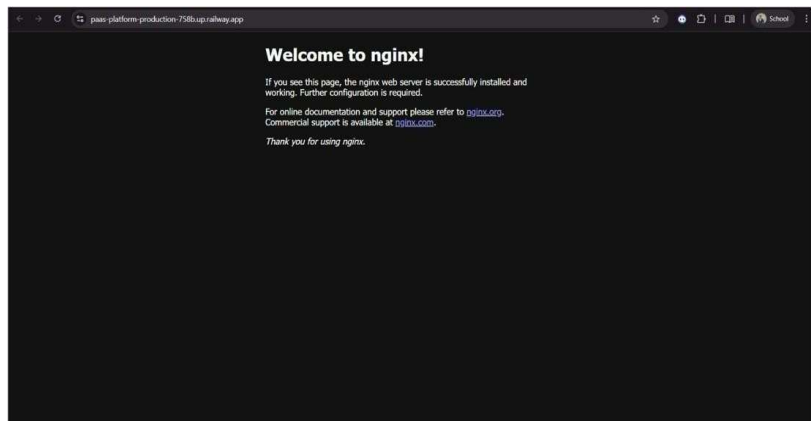
Step 4: Configure Railway Deployment

1. Railway automatically detects the application
2. Runtime environment is created automatically
3. Build and deployment start without manual configuration



Step 5: Deploy Application

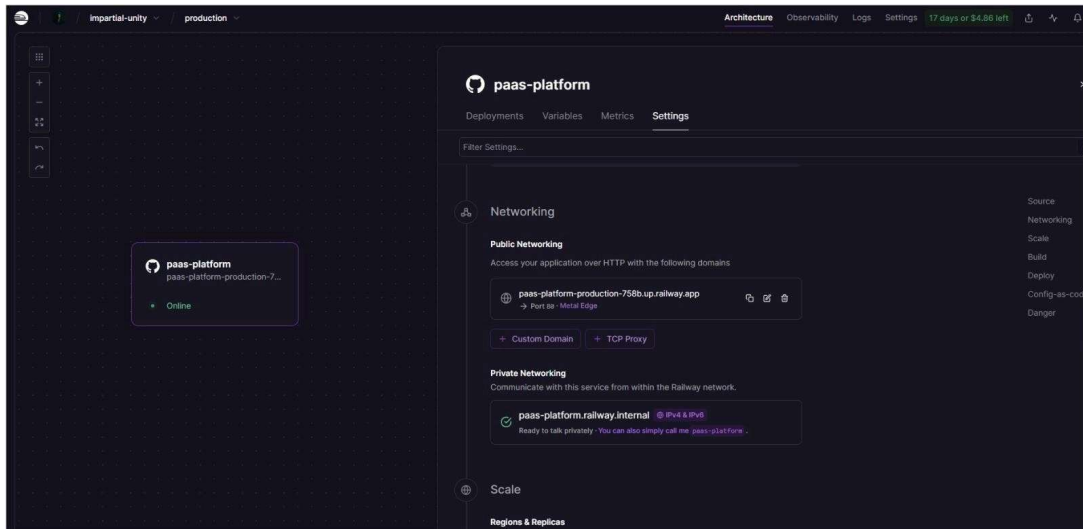
1. Railway builds the application
2. Application is deployed automatically
3. Deployment logs are displayed in dashboard



Step 6: Access Application

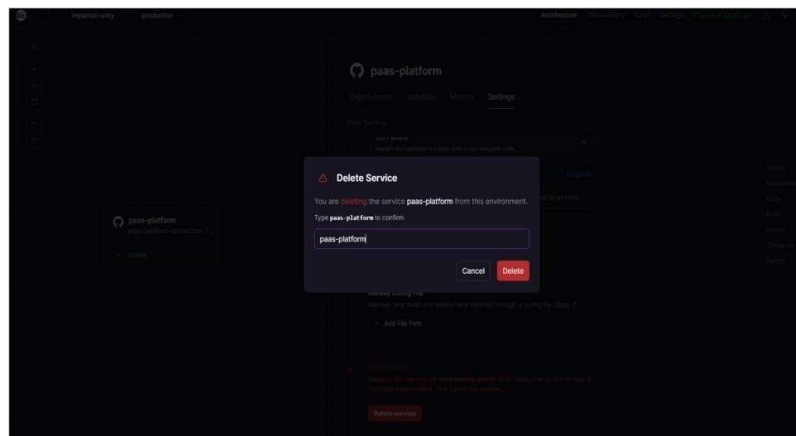
1. Click **View Deployment**
2. Open the generated public URL
3. Output displayed in browser:

Hello from Public PaaS using Railway Cloud



Step 7: Service Monitoring

1. Architecture view shows **paas-platform** service
2. Status indicator is **Online (Green)**
3. Replicas set to **1**
4. Restart policy set to **On failure**



Step 8 : Delete Service(Optional)

1. Go to settings → **Danger Zone**
2. Click **Delete Service**
3. Enter Service name pass- platform to confirm
4. Click **Delete**
5. Service is removed from Railway environment

Conclusion:-

Practical-8

Aim:- Implementation of Software as a Service (SaaS) in Private Cloud Environment

1. Aim of the Practical

To implement Software-as-a-Service (SaaS) in a Private Cloud environment using Docker containerization.

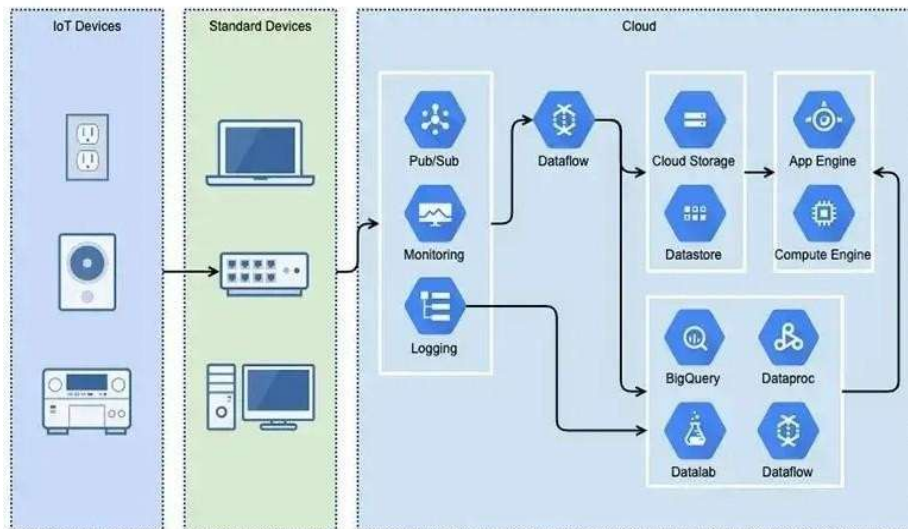
2. Introduction

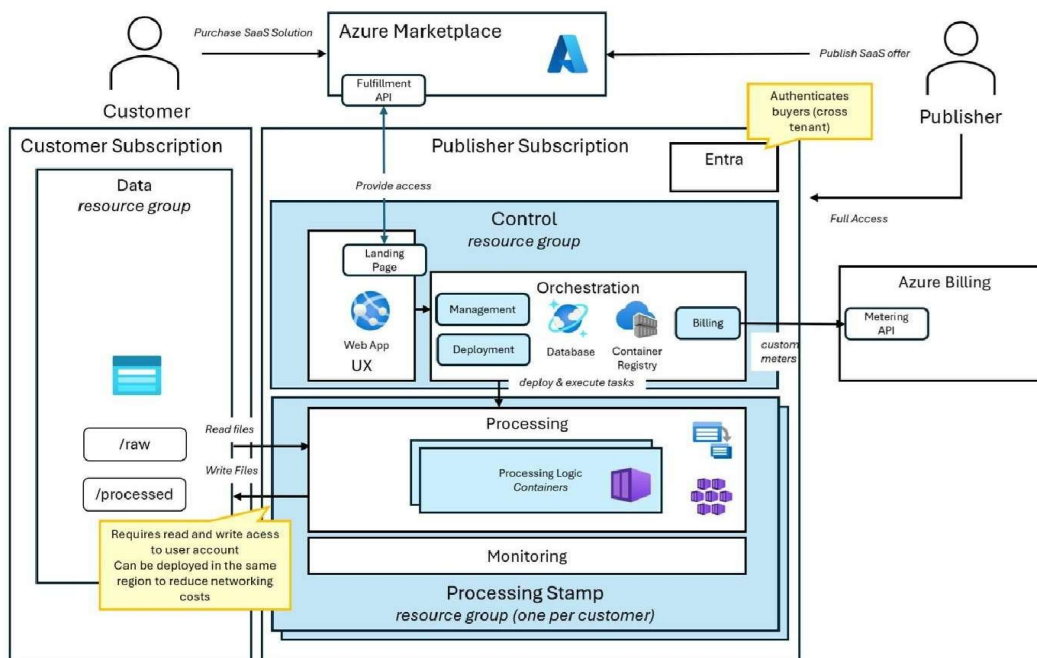
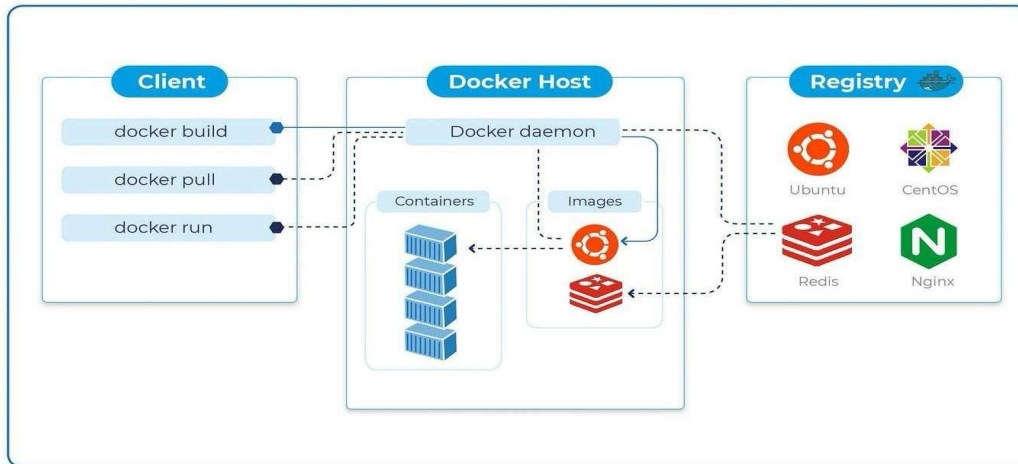
Software as a Service (SaaS) is a cloud computing model where software applications are hosted on cloud infrastructure and accessed via a web browser.

In a Private Cloud environment, infrastructure is managed internally using virtualization or containerization tools such as Docker.

In this experiment, a Flask web application is deployed using Docker to simulate SaaS in a private cloud.

3. Architecture of Private SaaS





Architecture Layers:

- User Layer – Web browser
- Application Layer – Flask SaaS App
- Container Layer – Docker
- Infrastructure Layer – Local Private Machine

4. Requirements

- Laptop/Desktop
- Docker Desktop Installed
- Python
- Internet (for image pull)

5. Step-by-Step Implementation

Step 1: Install Docker

Download and install:
Docker Desktop
Verify:
`docker --version`

Step 2: Create Project Folder

```
mkdir saas-private-cloud  
cd saas-private-cloud
```

Step 3: Create Application File

```
touch app.py  
Add:  
from flask import Flask  
app = Flask(__name_)  
@app.route("/")  
def home():  
    return "Welcome to SaaS Application hosted on Private Cloud!"  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

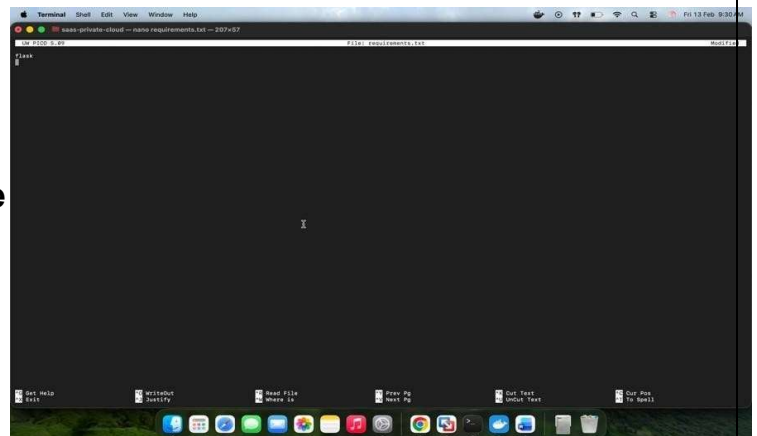
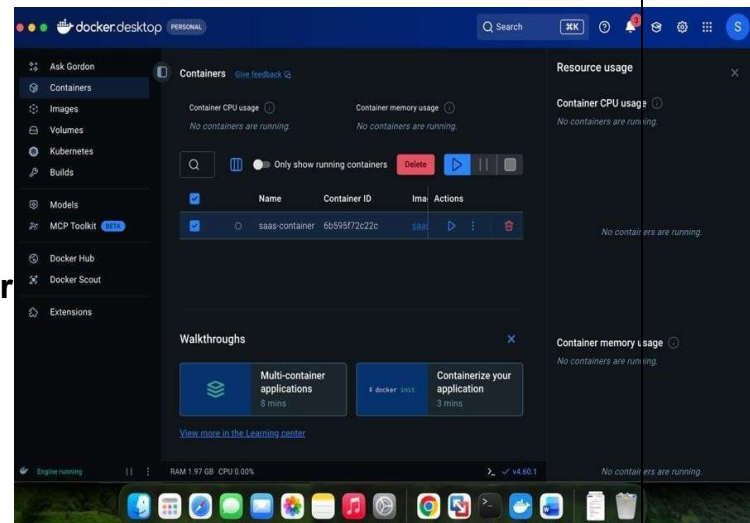
Step 4: Create requirements.txt

```
touch requirements.txt  
Add:  
flask
```

Step 5: Create Dockerfile

```
touch Dockerfile
```

Add:



```
FROM python:3.9

WORKDIR /app
COPY requirements.txt .
RUN pip install -r
requirements.txt COPY . .
CMD ["python", "app.py"]
```

Step 6: Build Docker Image

```
docker build -t saas-app .
```

Step 7: Run SaaS Container

(Use safe port like 8000)

```
docker run -d -p 8000:5000 --name saas-container saas-app
```

Step 8: Access Application

Open browser:

<http://localhost:8000>

t:8000 Output:

Welcome to SaaS Application hosted on Private Cloud!



6. Service Monitoring

Check running container:

```
docker ps
```

View logs:

```
docker logs saas-container
```

Status: Up

Port Mapping: 8000 → 5000

7. Stop and Remove Service

```
docker stop saas-
```

```
container docker rm
```

```
saas-container
```

Conclusion:-

Practical-9

Aim:- Implementation of Software as a Service (SaaS) in Public Cloud Environment

Aim

To implement Software as a Service (SaaS) in a Public Cloud environment using Render and deploy a web-based application accessible through a public URL.

Introduction

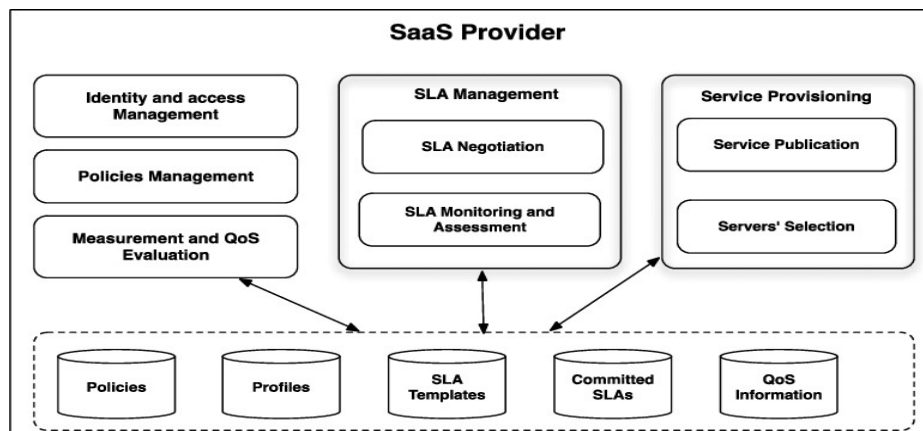
Software as a Service (SaaS) is a cloud computing model where applications are hosted on cloud infrastructure and delivered over the internet.

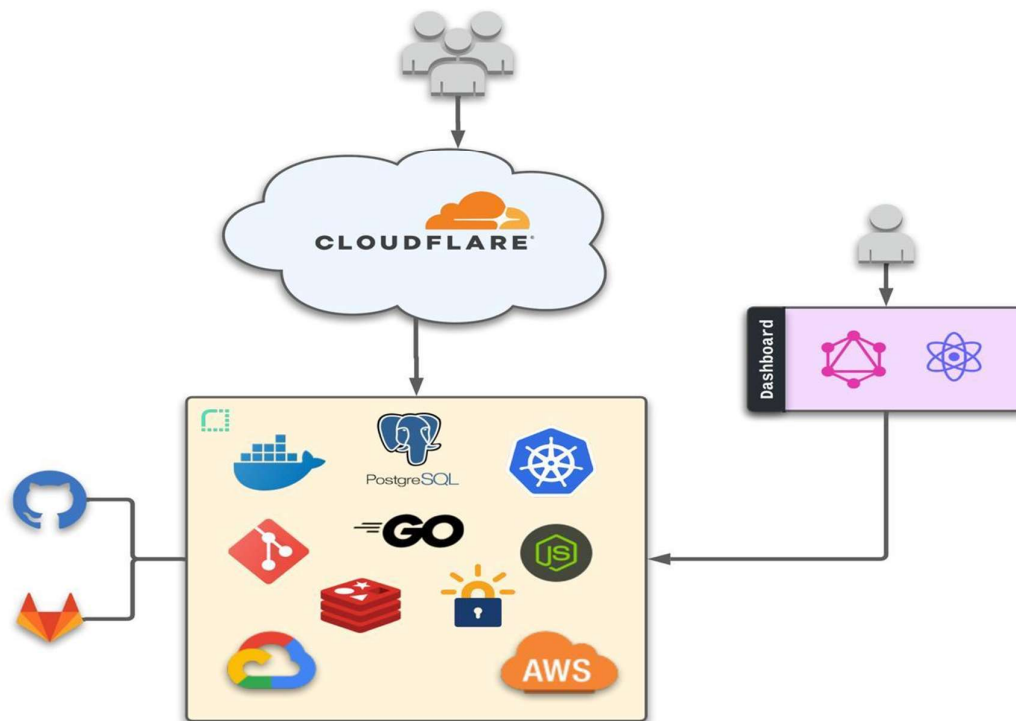
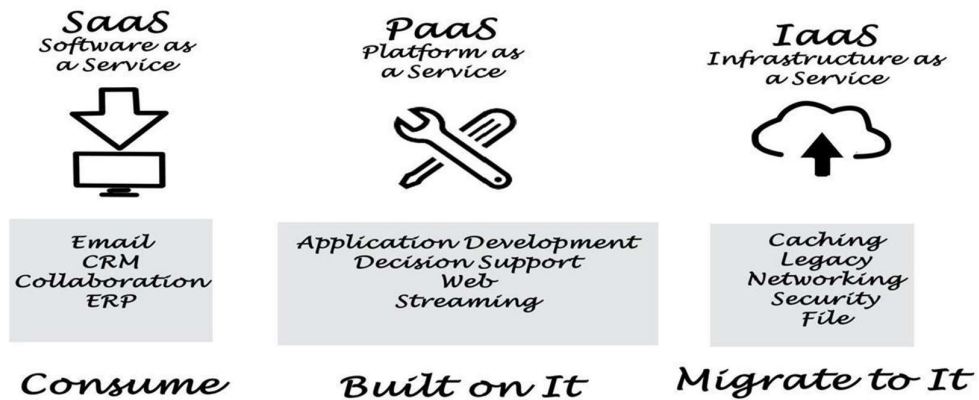
In Public Cloud SaaS:

- Infrastructure is managed by provider
- Application is accessible globally
- No local server management required

In this practical, we deploy a Flask web application using Render.

Architecture of Public SaaS





Layers:

- User Layer – Browser
- Application Layer – Flask SaaS App
- Cloud Platform – Render
- Infrastructure – Public Cloud Servers

Requirements

- Laptop/Desktop
- GitHub Account
- Render Account
- Internet Connection

Step-by-Step Implementation

Step 1: Create GitHub Repository

1. Go to <https://github.com>
2. Click **New Repository**
3. Name it: `saas-render`
4. Click Create

Step 2: Create Project Files Locally

Create `app.py`
`touch app.py`

ADD :

```
From flask import Flask  
App= Flask(__name__)
```

```
@app.route("/")  
def home():  
    return "Hello from SaaS in Public Cloud using Render"
```

```
if __name__ == "__main__":
```

```
app.run(host="0.0.0.0", port=10000)
```

Create `requirements.txt`

```
touch requirements.txt
```

Add:

flask
gunicorn

Create `render.yaml` (Optional but Professional)

```
touch render.yaml
Add:
services:
- type: web
  name: saas-render
runtime:python
  buildCommand: "pip install -r
  requirements.txt" startCommand:
  "gunicorn app:app"
```

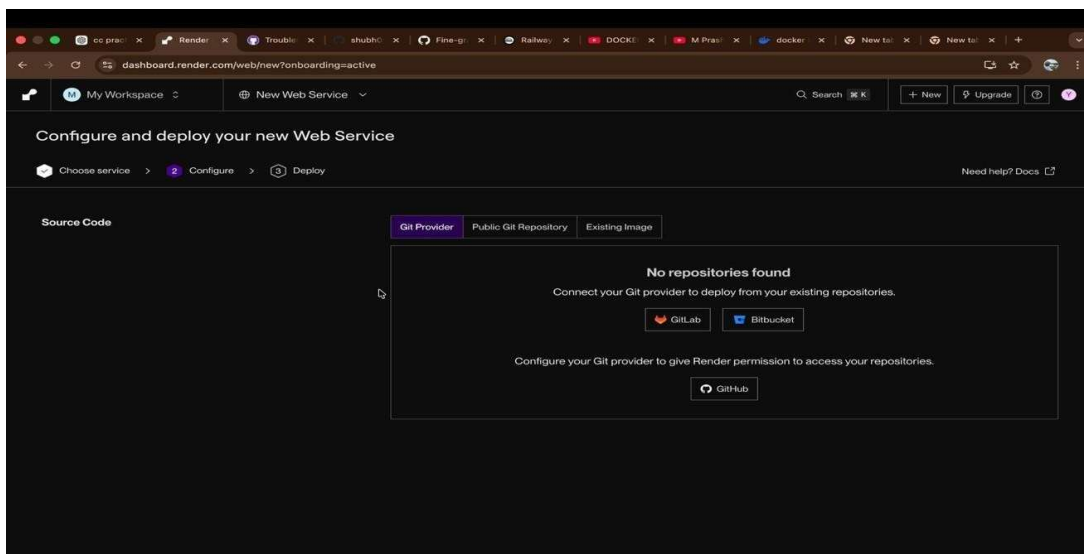
Step 3: Push to GitHub

```
git commit -m "Initial SaaS Render Deployment"
git remote add origin
https://github.com/YOUR_USERNAME/saas-render.git git
branch -M main
git push -u origin main
```

Step 4: Create Render Account

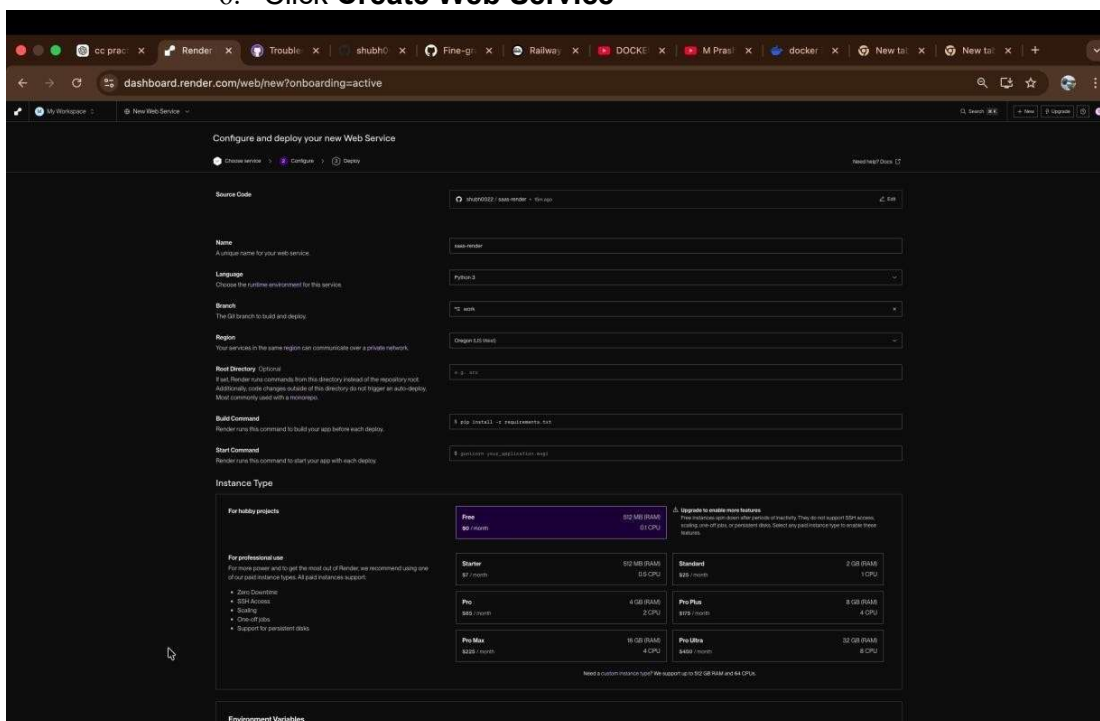
Go to: <https://render.com>

Login using GitHub



Step 5: Deploy on Render

1. Click **New +**
2. Select **Web Service**
3. Connect GitHub repository
4. Select `saas-render`
5. Choose:
 - Runtime → Python
 - Build Command → `pip install -r requirements.txt`
 - Start Command → `gunicorn app:app`
6. Click **Create Web Service**

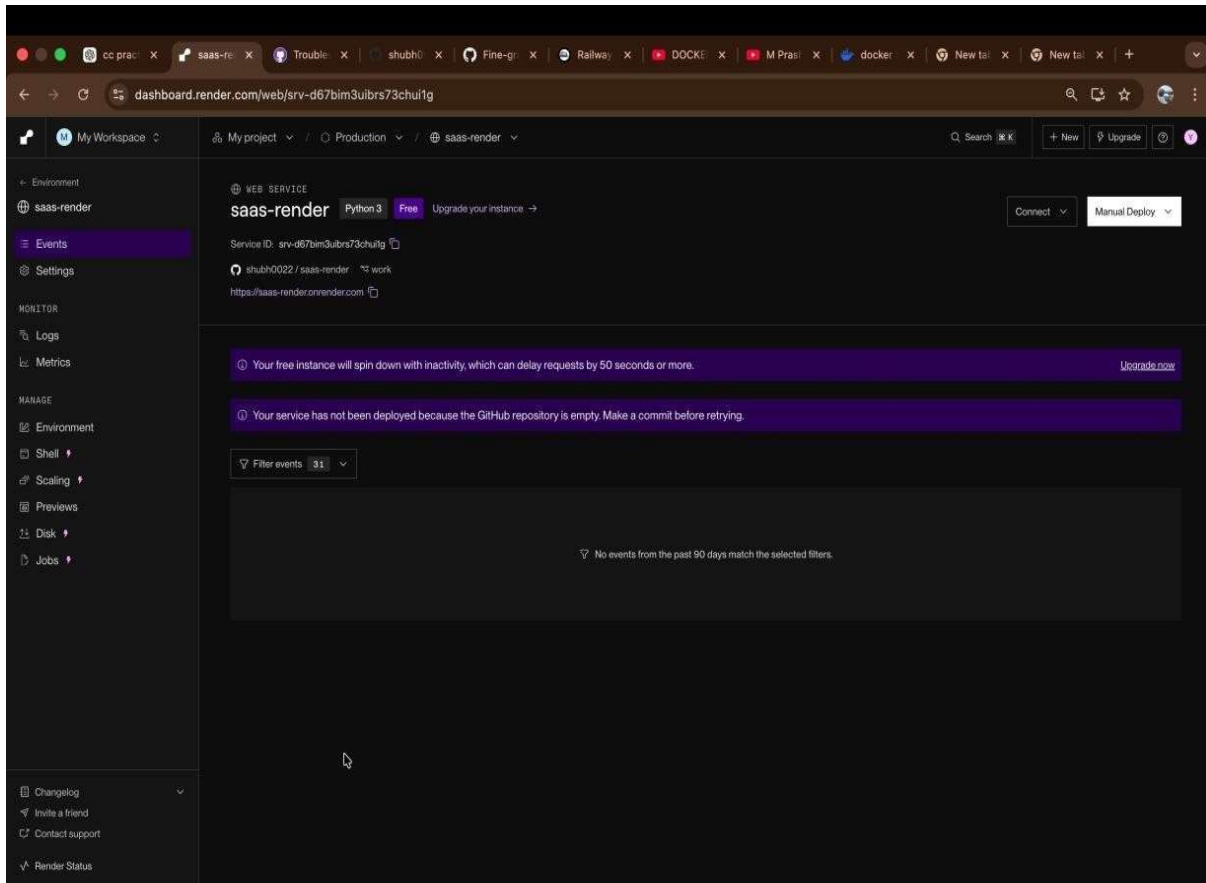


Step 6: Wait for Deployment

Render will:

Install dependencies
Build application
Deploy to public cloud

Status → Live (Green)

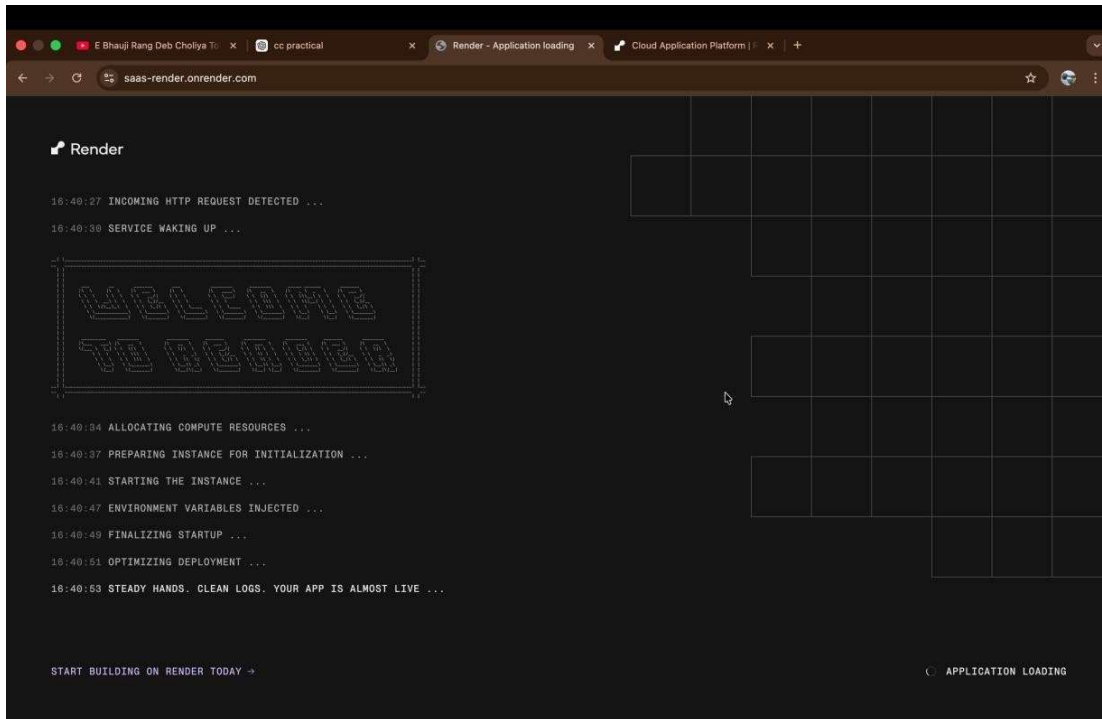


Step 7: Access Application

Click provided URL like:

<https://saas-render.onrender.com>
Output:

Hello from SaaS in Public Cloud using Render



Conclusion:-

Practical-10

Aim:- Implementation of Storage as a Service (SaaS) using Google Drive

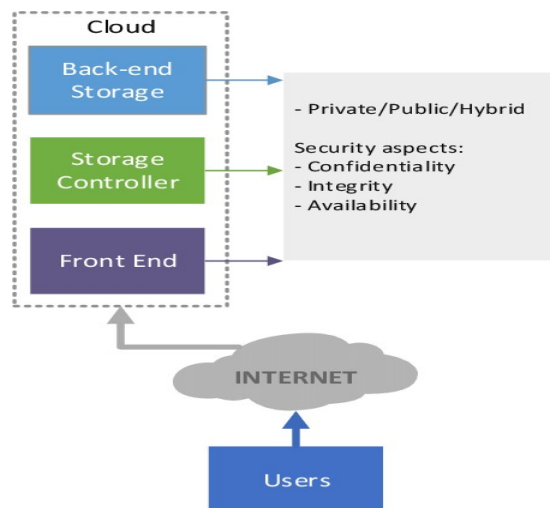
Introduction

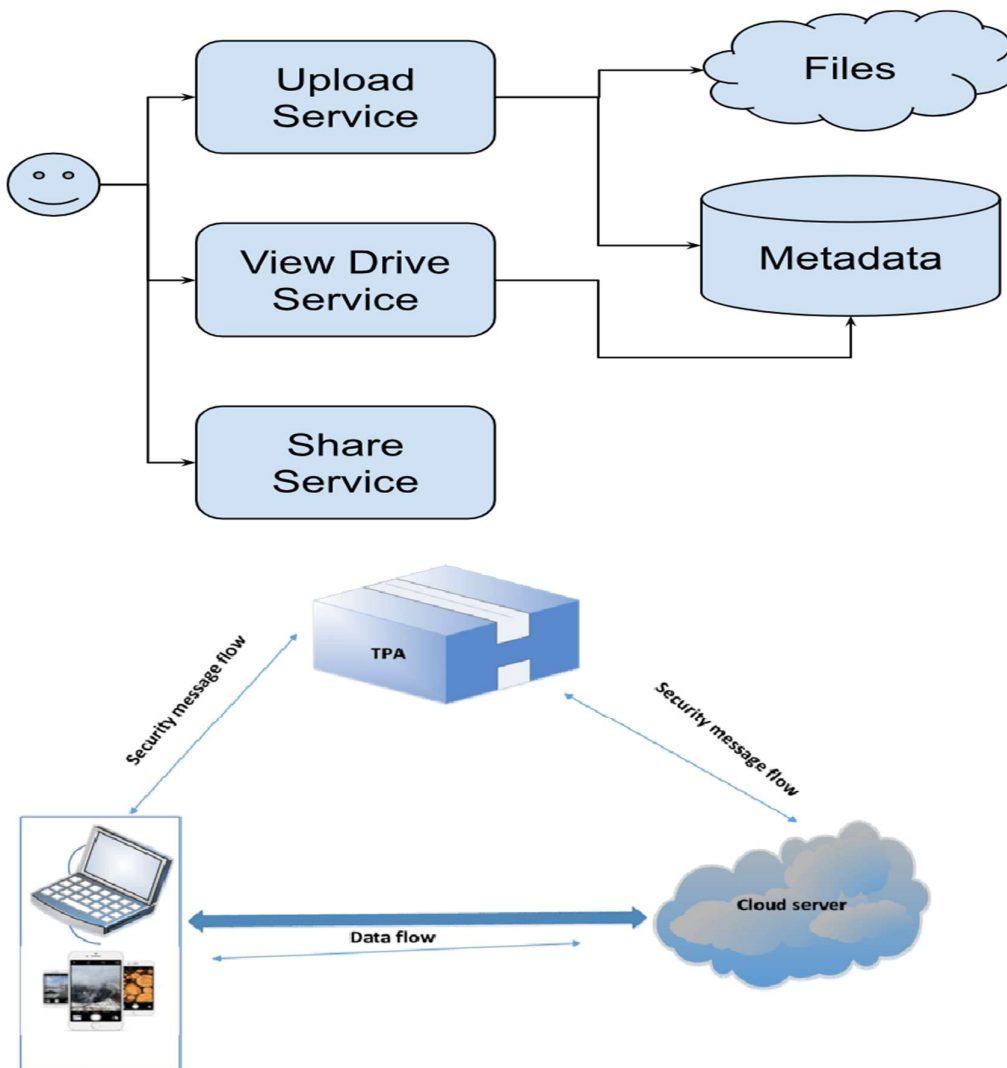
Storage as a Service (STaaS) is a cloud computing model where storage resources are provided over the internet by a cloud provider.

Google Drive allows users to:

- Upload files
- Store data in cloud
- Access data from anywhere
- Share files via public links

Architecture of SaaS





Requirements

- Laptop/Desktop
- Internet Connection
- Google Account

Step-by-Step Implementation

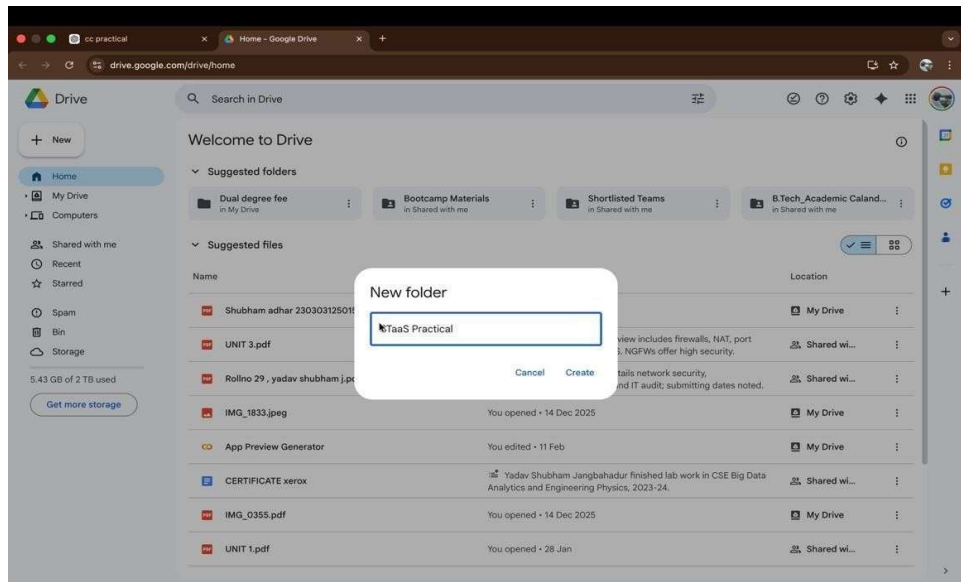
STEP 1: Open Google Drive

Go to: <https://drive.google.com>

Login using your Google account

STEP 2: Create New Folder (Optional)

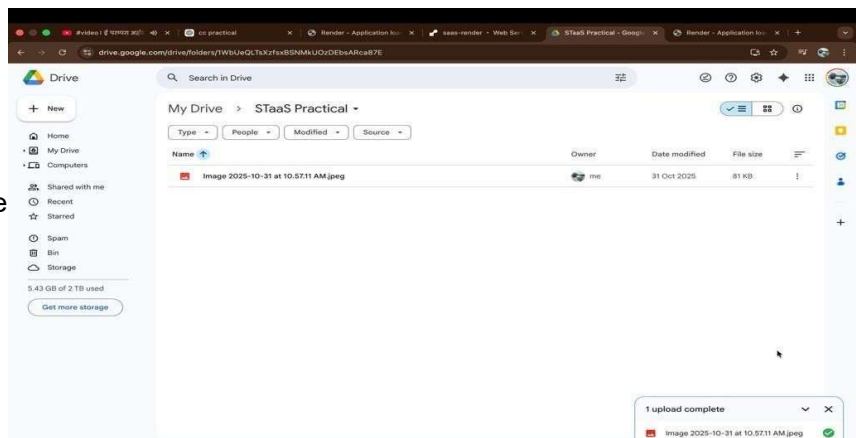
1. Click **+ New**
2. Select **Folder**
3. Name it:



STEP 3: Upload File to Cloud Storage

1. Click **+ New**
2. Select **File upload**
3. Choose any file (PDF/Image/Text)
4. Wait for upload to complete

File is now stored in cloud



Enrollment No: 2303031250067

STEP 4: Access File from Cloud

1. Double click uploaded file
2. It opens in browser

This proves cloud-based storage access.

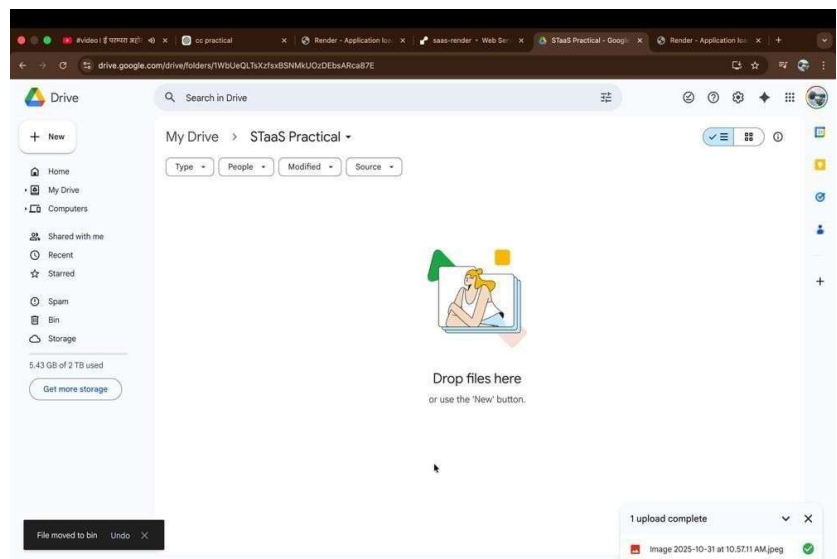
STEP 5: Share File (Demonstration of Cloud Access)

- 1.Right click file
- 2.Click **Get Link**
- 3.Change access to:

Anyone with the link →
Viewer

- 1.Copy link
- 2.Open in new browser tab

File opens successfully



STEP6:Delete (Optional)

Right click → Delete

Conclusion:-